

HOME SOC

Definitive Build Manual

Complete guide to build, operate, secure, and rebuild your Home Security Operations Centre from scratch

Proxmox VE 8.x · pfSense CE · Pi-hole · Wazuh 4.14 · Suricata IDS (Inline) · Tailscale VPN

Author: Samhith Challa

Version 2 · April 2026 · 21 Chapters · Complete Build Guide

PORTFOLIO DOCUMENT — PUBLIC EDITION

Network addresses and machine-specific identifiers have been generalised for public release.

Table of Contents

PART I — FOUNDATIONS & ARCHITECTURE

- Chapter 1 — Key Concepts & Glossary
- Chapter 2 — System Architecture & Design

PART II — HARDWARE & HOST SETUP

- Chapter 3 — Hardware Preparation
- Chapter 4 — Proxmox VE Installation
- Chapter 5 — Proxmox Network Bridges

PART III — NETWORK LAYER

- Chapter 6 — pfSense — Gateway & Firewall
- Chapter 7 — Access Point Configuration

PART IV — SECURITY SERVICES

- Chapter 8 — Pi-hole — DNS Filtering
- Chapter 9 — DNS Enforcement
- Chapter 10 — Tailscale — Secure Remote Access
- Chapter 11 — Suricata — Intrusion Detection System (Inline Bridge Mode)

PART V — SIEM LAYER

- Chapter 12 — Wazuh — SIEM Platform
- Chapter 13 — Log Integration Pipeline
- Chapter 14 — Wazuh Detection Rules

PART VI — STORAGE & OPERATIONS

- Chapter 15 — Storage Architecture
- Chapter 16 — Alerts & Notifications
- Chapter 17 — Security Hardening
- Chapter 18 — Daily Operations Runbook

PART VII — REBUILD & RECOVERY

- Chapter 19 — Full Rebuild Checklist
- Chapter 20 — Disaster Recovery Runbook

PART VIII — EXPANSION ROADMAP

- Chapter 21 — Future Enhancements

APPENDICES

- Appendix A — Master IP & VM Reference Table
- Appendix B — All Config Files — Ready to Copy
- Appendix C — Validation Commands Quick Reference
- Appendix D — Troubleshooting Quick Reference

Chapter 1 — Key Concepts & Glossary

Every technical term used in this manual is defined here before you need them. Read this chapter first — it will make every subsequent chapter significantly easier to follow.

1.1 What Is a Home SOC?

A Security Operations Centre (SOC) is a centralised system that monitors, detects, and responds to cyber threats. Enterprise SOC's employ teams of analysts working in shifts around the clock, using expensive commercial tools and dedicated hardware.

This Home SOC achieves the same outcome using entirely free open-source software running on a single laptop — protecting every device on a home network 24 hours a day, automatically, without any manual intervention.

WHY DOES A HOME NEED A SOC? Modern home networks contain smartphones, smart TVs, laptops, baby monitors, gaming consoles, and IoT devices — all potential attack targets. Your ISP router offers zero visibility into what is actually happening on your network. A Home SOC gives you firewall control, DNS filtering, intrusion detection, and centralised alerting — the same protection companies pay millions for, running on hardware you already own.

1.2 Master Glossary of Terms

The following terms are used throughout this manual. Understanding these before proceeding will prevent confusion during implementation.

Hypervisor

Software that lets one physical computer run multiple virtual computers simultaneously. Proxmox VE is the hypervisor in this setup — it is the hotel manager assigning rooms (CPU, RAM, disk) to guests (virtual machines). Without the hypervisor, none of the other components can run.

Virtual Machine (VM)

A fully isolated computer that exists only as software inside the hypervisor. Has its own operating system, CPU allocation, RAM allocation, and virtual disk. Behaves exactly like a physical machine. pfSense, Suricata, and Wazuh all run as VMs in this setup.

LXC Container

A lighter alternative to a VM. Shares the host operating system kernel but keeps processes isolated from each other and from the host. Starts in under a second and uses far less memory than a full VM. Pi-hole and Tailscale run as LXC containers.

Network Bridge (vmbri)

A virtual network switch that exists entirely in software inside Proxmox. Physical network cards plug into bridges. VMs connect to bridges to communicate. This setup uses three bridges: vmbri0 (WAN-facing), vmbri1 (LAN client-facing), and vmbri2 (internal, connects Suricata to pfSense).

vmbri2 — The V2 Innovation

An internal Proxmox bridge with no physical network card attached. Used to connect Suricata's output interface to pfSense's LAN interface, completing the inline inspection path. Without vmbri2, the inline architecture cannot be built. This bridge was the key change between V1 and V2 of this build.

Inline IDS

An Intrusion Detection System placed physically in the traffic path so that ALL packets pass through it. In contrast, a passive IDS monitors a copy of traffic. Suricata in this setup is inline — it acts as a transparent bridge between LAN clients and pfSense, meaning traffic enters on one interface, gets inspected, and exits on the other interface. Devices have no idea Suricata exists.

WAN — Wide Area Network

The internet-facing side of the network. Your ISP provides the WAN connection. In this setup, the WAN subnet is referred to as [WAN_SUBNET] throughout all configuration files and commands.

LAN — Local Area Network

Your private home network. All your devices — phones, laptops, smart TVs, IoT devices — live on the LAN. In this setup, the LAN subnet is referred to as [LAN_SUBNET] throughout all configuration files and commands.

DHCP — Dynamic Host Configuration Protocol

Automatically assigns IP addresses to devices when they join the network. In this setup, pfSense is the ONLY DHCP server. No other device should ever be configured to hand out IP addresses — doing so creates a rogue DHCP server and breaks network routing.

DNS — Domain Name System

Translates human-readable domain names (google.com) into IP addresses that computers use (142.250.x.x). Pi-hole acts as the DNS server for the entire network. When a device asks for a malicious domain, Pi-hole returns 0.0.0.0 — the device never connects to the threat.

Firewall

Inspects network traffic and allows or blocks it based on rules. pfSense is the firewall in this setup. It enforces rules such as blocking all DNS except to Pi-hole, preventing devices from bypassing DNS filtering.

SIEM — Security Information and Event Management

Software that collects log data from many sources, correlates events across those sources, and generates alerts when suspicious patterns are detected. Wazuh is the SIEM in this setup. It receives logs from pfSense, Suricata, Pi-hole, Tailscale, Proxmox, and Windows endpoints simultaneously.

IDS — Intrusion Detection System

Software that monitors network traffic and raises alerts when it recognises known attack patterns — port scans, brute force attempts, exploit signatures, malware communication. Suricata is the IDS in this setup.

IPS — Intrusion Prevention System

An IDS that can also actively block traffic, not just alert on it. Suricata can operate in IPS mode (future enhancement — see Chapter 21). Currently configured as IDS only.

Syslog

A standard protocol for sending log messages from one device to another over the network using UDP port 514. pfSense sends its firewall log events to Wazuh via syslog.

Wazuh Agent

A small program installed on a monitored device that reads local log files and ships them to the Wazuh Manager. Agents are installed on the Suricata VM, Pi-hole LXC, Tailscale LXC, Proxmox host, and Windows laptop.

eve.json

Suricata's structured alert output file. Every IDS event Suricata detects is written to this file in JSON format. The Wazuh agent on the Suricata VM reads this file continuously and forwards every event to the SIEM for correlation and alerting.

Linux Bridge (br0)

A software bridge created inside the Suricata VM that connects both of its network interfaces (exxxx8 from vmbr1 and xxxxx9 to vmbr2). This makes Suricata transparent to all traffic — packets flow through the bridge without any device knowing Suricata is inspecting them.

NIC Offloading

Hardware acceleration features (GRO, LRO, TSO, GSO) that batch and modify network packets to improve throughput. These features MUST be disabled on Suricata's bridge interfaces — if active, they cause Suricata to see modified or reassembled packets rather than the original network frames, breaking detection accuracy.

Transparent Bridge

A network device that forwards traffic between two interfaces without modifying it and without any device being aware of its presence. Suricata's br0 bridge is fully transparent to all LAN devices and to pfSense.

Snapshot

A saved state of a VM or container at a specific point in time. Proxmox can create and restore snapshots in seconds. The golden rule: take a snapshot before every configuration change. If something breaks, restore the snapshot instantly.

NAT — Network Address Translation

Allows many private LAN devices to share one public IP address when accessing the internet. pfSense performs NAT. Also used in this setup to redirect all DNS queries to Pi-hole regardless of what DNS server a device is configured to use.

Gravity — Pi-hole

Pi-hole's compiled blocklist database. When you run 'pihole -g', Pi-hole downloads and compiles all configured blocklists into a single database used for DNS filtering. The Gravity database must be updated weekly to catch new malicious domains.

Chapter 2 — System Architecture & Design

This chapter describes how all components connect, how traffic flows through the system, and the design principles that make this architecture secure and maintainable.

V2 ARCHITECTURE CHANGE Suricata is now INLINE in the traffic path. All traffic from LAN clients passes through the Suricata VM bridge before reaching pfSense. This gives Suricata full packet visibility including all TCP SYN packets and connection attempts — something the V1 passive architecture could not achieve.

2.1 The Dependency Chain

The system is layered. Each layer depends on all layers below it. Always build bottom-up. Always troubleshoot bottom-up. If Layer 4 breaks, check Layers 1-3 before assuming Layer 4 is the problem.

Layer	Component	Depends On	If Missing — Impact
Layer 1 — Host	Proxmox VE	Physical hardware	Nothing can run
Layer 2 — Fabric	vmbr0 + vmbr1 + vmbr2	Proxmox installed	VMs have no network
Layer 3 — Inline IDS	Suricata (VM 101)	Bridges + vmbr2	Traffic bypasses inspection
Layer 4 — Gateway	pfSense (VM 100)	Suricata bridge path	No internet, no DHCP, no DNS
Layer 5 — DNS	Pi-hole (LXC 102)	pfSense routing	DNS fails, websites unreachable
Layer 6 — SIEM	Wazuh (VM 104)	All log sources	No alerts, blind monitoring
Layer 7 — VPN	Tailscale (LXC 103)	pfSense + Pi-hole	No remote access

2.2 Updated Network Architecture (V2 — Inline Suricata)

The following diagram shows the complete physical and logical topology of the V2 system. Every packet from every LAN device must pass through Suricata before reaching pfSense.

```
ISP (Internet)
|
v [Ethernet]
Home Router ([WAN_SUBNET].1)
| [Ethernet cable to laptop built-in NIC]
v
Proxmox Host ([WAN_SUBNET].10)
| [vmbr0 - WAN bridge]
v
pfSense WAN interface (vtnet0) <--- internet enters here

===== LAN SIDE =====

LAN Clients (phones, laptops, Wi-Fi devices)
| [USB Ethernet NIC → vmbr1]
v
Suricata VM - exxxxx8 (vmbr1 client-facing input)
|
| [br0 - Linux bridge INSIDE Suricata VM]
| *** ALL TRAFFIC INSPECTED HERE ***
|
v
Suricata VM - exxxxx9 (vmbr2 pfSense-facing output)
| [vmbr2 - internal bridge, no physical NIC]
```

```

v
pfSense LAN interface (vtnet1) <--- routes to internet

Pi-hole      ([LAN_SUBNET].4) - DNS server on vmbr1
Wazuh        ([LAN_SUBNET].7) - SIEM on vmbr1
Tailscale    ([LAN_SUBNET].5) - VPN router on vmbr1

```

NOTE KEY INSIGHT: Pi-hole, Wazuh, and Tailscale connect directly to vmbr1 — they do NOT pass through Suricata. Only traffic from external LAN client devices (phones, laptops) goes through the Suricata inline path. SOC infrastructure components communicate directly on the LAN bridge.

2.3 DNS Security Flow

Every DNS query from every device on the network follows this path:

```

Client device (phone, laptop, smart TV)
|
| DNS query: 'What is evil-malware-c2.com?'
| (passes through Suricata br0 - inspected for DNS anomalies)
v
Pi-hole ([LAN_SUBNET].4)
|
+-- Domain in blocklist? --> YES --> Return 0.0.0.0 (blocked)
|
|                                     |
|                                     v
|                                     Wazuh alert generated
|
+-- Domain safe? -----> YES --> Forward to 1.1.1.1 or 8.8.8.8
|
|                                     v
|                                     Return real IP to client

```

Critically: even if a device tries to use a hardcoded DNS server (like 8.8.8.8 directly), pfSense's NAT redirect rule intercepts the query and routes it to Pi-hole anyway. No device can bypass this control.

2.4 Log Flow — SIEM Pipeline

Wazuh receives logs from six different sources simultaneously. Every event from every component flows into a single SIEM for correlation and alerting.

```

pfSense firewall events -----> [Syslog UDP 514] -----> +
|
Pi-hole DNS queries -----> [Wazuh Agent port 1514] --> |
|
Suricata IDS alerts (eve.json) --> [Wazuh Agent port 1514] --> | Wazuh Manager
|                                     | [LAN_SUBNET].7
Tailscale VPN events -----> [Wazuh Agent port 1514] --> |
|                                     | Apply detection
Proxmox system events -----> [Syslog] -----> | rules
|                                     | Generate alerts
Windows Laptop (Sysmon) -----> [Wazuh Agent port 1514] --> +
|
|                                     v
|                                     Wazuh Dashboard
|                                     + Telegram alerts

```

2.5 Design Principles

Inline IDS (V2 Architecture)

Suricata is in the traffic path, not a passive observer. Every single packet from every LAN device passes through Suricata before reaching pfSense. This gives complete packet visibility including connection attempts, not just established flows. This was the critical fix from V1 where Suricata was passive and could not see any traffic.

Single Routing Authority

pfSense is the only router and the only DHCP server. All internet-bound traffic passes through pfSense after being inspected by Suricata. No other device performs routing or assigns IP addresses.

DNS as a Security Layer

All DNS is forced through Pi-hole via pfSense firewall rules and NAT redirect. Malware almost always uses DNS to locate command-and-control servers — centralising and filtering DNS makes malware communication visible and blockable.

Role Separation

Each component does exactly one job. Suricata inspects packets. Pi-hole filters DNS. pfSense routes traffic and enforces firewall rules. Wazuh analyses logs and generates alerts. This separation means failures are isolated — if one component fails, the others continue operating.

Snapshot-First Discipline

Every major configuration change is preceded by taking a Proxmox snapshot. If a change breaks something, the snapshot can be restored in seconds with no data loss. Never make a change without a current snapshot.

Deterministic Rebuild

Every component in this system is defined by a configuration file that is documented in this manual. The entire system can be rebuilt from scratch on bare hardware using only this document. No step depends on undocumented tribal knowledge.

2.6 Master IP Address Table

Every infrastructure component has a permanent static IP address. Note: Specific addresses have been replaced with subnet labels in this public edition. [WAN_SUBNET] = your WAN-side network. [LAN_SUBNET] = your LAN-side private network.

Device / Service	IP Address	Type	Role
ISP Router	[WAN_SUBNET].1	Physical	Gateway to internet
Proxmox Host	[WAN_SUBNET].10	Physical	Hypervisor management UI
pfSense WAN	DHCP from ISP	VM 100	Firewall WAN interface
pfSense LAN	[LAN_SUBNET].1	VM 100	Gateway · DHCP server · DNS enforcer
Access Point	[LAN_SUBNET].2	Physical	Wi-Fi + LAN switch (AP mode only)
Pi-hole	[LAN_SUBNET].4	LXC 102	DNS filter · Ad + malware blocker
Tailscale	[LAN_SUBNET].5	LXC 103	VPN subnet router
Suricata IDS	[LAN_SUBNET].6	VM 101	Inline network IDS (bridge mode)
Wazuh SIEM	[LAN_SUBNET].7	VM 104	SIEM · Log analysis · Alerting
LAN Clients (DHCP)	[LAN_SUBNET].10–99	Dynamic	Phones, laptops, smart devices
Infra Reserved	[LAN_SUBNET].1–9	Static	Never assigned to DHCP pool
vmbr2	Internal bridge only	Bridge	Suricata output → pfSense LAN (V2)

NOTE Infrastructure IPs [LAN_SUBNET].1 through .9 are permanently reserved and must never be included in the DHCP pool. pfSense DHCP pool starts at .10 and ends at .99.

Chapter 3 — Hardware Preparation

CRITICAL If you skip this chapter, network design breaks. pfSense fails to route. The entire SOC becomes unstable. Hardware preparation is not optional — it is the foundation everything else sits on.

3.1 Host System Overview

The host system is the physical laptop that runs everything. Proxmox VE is installed directly on this laptop, replacing any previous operating system completely. All virtual machines and containers — pfSense, Suricata, Wazuh, Pi-hole, Tailscale — run inside this single physical machine as isolated VMs and containers.

The laptop remains always-on, running as a server 24/7. This requires specific BIOS configuration and OS-level settings to prevent it from sleeping or hibernating.

```
Physical Laptop (Host System)
|
+-- Proxmox VE 8.x (Hypervisor)
|
+-- VM 100: pfSense CE (Firewall/Router) → vmbr0 (WAN) + vmbr2 (LAN)
+-- VM 101: Suricata (Inline IDS bridge) → vmbr1 (in) + vmbr2 (out)
+-- VM 104: Wazuh 4.14 (SIEM)           → vmbr1
+-- LXC 102: Pi-hole (DNS filter)        → vmbr1
+-- LXC 103: Tailscale (VPN router)      → vmbr1 + vmbr0
```

3.2 Hardware Requirements

Component	Minimum	Recommended	Why This Matters
CPU	Intel/AMD with VT-x/AMD-V	4+ cores	Virtualisation extensions required. Without VT-x/AMD-V, VMs cannot run.
RAM	8 GB	12+ GB	Wazuh indexer alone consumes 4-6 GB. 8 GB is the absolute minimum. 12 GB allows comfortable operation with all VMs running simultaneously.
Storage	128 GB SSD	256+ GB SSD	Fast I/O required for Wazuh log indexing. HDD will cause significant performance issues. SSD is non-negotiable.
NIC 1	Built-in Ethernet	Gigabit built-in	WAN connection to home router. Used for vmbr0.
NIC 2	USB Ethernet adapter	USB 3.0 Gigabit	LAN client side. Mandatory for pfSense to have a separate LAN interface. Used for vmbr1.

3.3 Network Interface Design (V2)

The V2 architecture requires three network bridges and two physical NICs.

Physical Interface	Proxmox Bridge	Role
Built-in Ethernet (e.g. eno1)	vmbr0	WAN — connects to ISP router. Proxmox management UI also accessible here.
USB Ethernet (e.g. enxXXX)	vmbr1	LAN client side — LAN devices, Pi-hole, Wazuh, Tailscale connect here. Also Suricata's input interface.
(No physical NIC)	vmbr2	Internal only — Suricata output to pfSense LAN. No physical NIC attached. V2 addition.

NOTE V1 used only two bridges (vmbr0 WAN, vmbr1 LAN). V2 adds vmbr2 — an internal bridge with no physical NIC — to complete the inline inspection path through the Suricata VM. This is the architectural change that made full packet visibility possible.

3.4 BIOS Configuration

These settings must be configured before installing Proxmox. Access BIOS by pressing F2, F10, F12, or Delete during laptop startup (varies by manufacturer).

Setting	Required Value	Why — Consequence If Wrong
Virtualisation (VT-x / AMD-V)	ENABLED	Required for all VMs to run. If disabled, Proxmox installs but no VMs can start.
Boot Mode	UEFI	Proxmox installer requires UEFI. Legacy BIOS may cause bootloader failures.
Secure Boot	DISABLED	Secure Boot can block the Proxmox bootloader from loading. Must be off.
Wake on LAN	ENABLED (if available)	Optional. Allows remote power-on if the laptop loses power.
CPU Power Management	Performance or Balanced	Power Saver mode can throttle CPU and cause VM instability.

3.5 Physical Wiring

The physical cable connections must be correct before starting. An incorrect connection — especially plugging into the AP's WAN port instead of a LAN port — will create double NAT and break the entire network.

```
ISP (Internet)
| [coaxial or DSL]
v
Home Router / Modem ([WAN_SUBNET].1)
| [Ethernet cable]
v
Laptop BUILT-IN NIC (e.g. eno1) --> vmbr0 (WAN bridge)

Laptop USB ETHERNET ADAPTER --> vmbr1 (LAN client bridge)
| [Ethernet cable]
v
Access Point ([LAN_SUBNET].2) [configured in AP mode]
|
+-- Wi-Fi --> Phones, tablets, laptops (wireless)
+-- LAN port 1 --> Wired desktop (if any)
+-- LAN port 2 --> Additional wired devices
```

CRITICAL CRITICAL WIRING RULE: Connect the Ethernet cable from the USB NIC to a LAN port on the access point — NEVER to the WAN/Internet port. Using the WAN port re-enables the access point's built-in router functions and creates a rogue DHCP server that will break IP assignment for all devices.

3.6 Power and Thermal Considerations

Running a laptop as a 24/7 server creates thermal and power considerations that must be managed:

- Ensure the laptop has adequate ventilation — do not place it flat on a soft surface that blocks vents
- Some laptop models does NOT support auto power-on after power loss. After a power cut: press the power button manually → Proxmox boots → all VMs and LXC's with onboot=1 start automatically within 2-3 minutes
- Consider a UPS (Uninterruptible Power Supply) to protect against brief power outages
- Monitor CPU temperature periodically — sustained high temperature can cause thermal throttling and instability

Chapter 4 — Proxmox VE Installation

CRITICAL Installing Proxmox ERASES EVERYTHING on the target disk. All existing data will be destroyed. Back up anything important BEFORE proceeding.

4.1 What Is Proxmox VE?

Proxmox VE (Virtual Environment) is a Type-1 Hypervisor — it runs directly on physical hardware with no underlying operating system beneath it. Unlike running VirtualBox inside Windows, Proxmox IS the operating system. It manages all VMs and LXC containers through a web-based UI accessible at [https://\[WAN_SUBNET\].10:8006](https://[WAN_SUBNET].10:8006) from any device on the network.

Proxmox is free and open source. The enterprise subscription is optional — this build uses the free no-subscription repository.

4.2 Preparing the Installer

Download the latest Proxmox VE 8.x ISO from: <https://www.proxmox.com/en/downloads/proxmox-virtual-environment/iso>

Tool	Platform	Method
Rufus	Windows	Open Rufus → Select ISO → Partition scheme: GPT → Select target USB → Write in DD Image mode → Start
BalenaEtcher	Win/Mac/Linux	Open Etcher → Flash from file → Select ISO → Select target USB → Flash
dd command	Linux/Mac terminal	dd if=proxmox-ve_8.x.iso of=/dev/sdX bs=1M status=progress (replace sdX with your USB device)

4.3 Full Installation Walkthrough

Boot the laptop from USB (press F12 or F2 during startup to access boot menu). The Proxmox graphical installer will load. Follow these steps:

Step	Action	Expected Result
1	Select 'Install Proxmox VE (Graphical)' from boot menu	Blue graphical installer screen appears
2	End User License Agreement → I agree	Proceeds to disk selection
3	Select internal SSD as target disk. Filesystem: ext4	Disk selected, no errors shown
4	Set country, timezone, keyboard layout	Timezone displayed correctly — critical for Wazuh log timestamps
5	Set strong root password (write this down — losing it locks you out). Email: any valid address	Password accepted, no error
6	Management IP: [WAN_SUBNET].10/24. Gateway: [WAN_SUBNET].1. Hostname: pve.localdomain. DNS: 8.8.8.8	Network configuration shown correctly
7	Click Install. Wait 3-5 minutes for installation to complete.	Progress bar fills, installation completes
8	Remove USB drive when prompted. System reboots.	System reboots to Proxmox boot screen

9	From another device on the same network: open <code>https://[WAN_SUBNET].10:8006</code> in browser. Accept the self-signed certificate warning.	Proxmox login page loads. Login: root / your password
---	---	---

4.4 Repository Fix (Required — Do This Before Anything Else)

By default, Proxmox is configured to use the enterprise repository which requires a paid subscription. Running 'apt update' without fixing this produces 401 Unauthorized errors. Fix it immediately after first login:

```
# Step 1: Disable enterprise repository
nano /etc/apt/sources.list.d/pve-enterprise.list
# Add a # at the very start of the line beginning with 'deb'
# So it reads: # deb https://enterprise.proxmox.com/debian/pve ...
# Save: Ctrl+O, Enter, Ctrl+X

# Step 2: Also disable Ceph enterprise repo if present
nano /etc/apt/sources.list.d/ceph.list
# Comment out any enterprise lines the same way

# Step 3: Add free no-subscription repository
nano /etc/apt/sources.list
# Add this line at the bottom of the file:
deb http://download.proxmox.com/debian/pve bookworm pve-no-subscription
# Save: Ctrl+O, Enter, Ctrl+X

# Step 4: Update and upgrade
apt update && apt full-upgrade -y
# Expected output: no 401 Unauthorized errors
# May take several minutes to download upgrades
```

4.5 Laptop Server Fixes (Critical)

A laptop running as a server will sleep, suspend, or hibernate by default — killing all VMs instantly. These fixes permanently prevent that:

```
# Fix 1: Disable lid-close sleep (so closing lid doesn't suspend the system)
nano /etc/systemd/logind.conf
# Find and set (or add) these three lines:
HandleLidSwitch=ignore
HandleLidSwitchExternalPower=ignore
HandleLidSwitchDocked=ignore
# Save file, then apply immediately:
systemctl restart systemd-logind

# Fix 2: Mask all sleep targets permanently (belt-and-suspenders approach)
systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target
# Verify all are masked:
systemctl status sleep.target
# Expected: 'Loaded: masked'

# Fix 3: Screen blank (not suspend) — saves power without sleeping
nano /etc/default/grub
# Find GRUB_CMDLINE_LINUX_DEFAULT and change to:
GRUB_CMDLINE_LINUX_DEFAULT="quiet consoleblank=60"
# Apply GRUB change:
update-grub
reboot
# After reboot: system should no longer sleep under any condition
```

4.6 Validation

After reboot, verify Proxmox is healthy before proceeding:

```
# Check Proxmox version
pveversion
# Expected: pve-manager/8.x.x/...

# Verify repositories are working (no 401 errors)
apt update
# Expected: lines showing 'Get:' with package lists, no errors

# Verify sleep is permanently disabled
systemctl status sleep.target
# Expected: 'Loaded: masked'

# Access UI from another device
# https://[WAN_SUBNET].10:8006
# Expected: Proxmox login page loads
```

Chapter 5 — Proxmox Network Bridges

NOTE V2 requires three bridges: vmbr0 (WAN), vmbr1 (LAN clients), vmbr2 (internal — no physical NIC). vmbr2 is the V2 addition that makes inline Suricata possible.

5.1 What Is a Network Bridge?

A network bridge (called vmbr in Proxmox) is a virtual network switch that exists entirely in software. Physical network cards plug into bridges. VMs connect to bridges to send and receive traffic through those physical cards.

Think of each bridge as a separate network switch in a rack. vmbr0 is your WAN switch (connected to the internet-facing cable). vmbr1 is your LAN switch (connected to client devices). vmbr2 is an internal patch panel between Suricata and pfSense — no cable needed because both are virtual.

5.2 Identifying Your Network Interface Names

Before editing the network configuration, you must know the exact names of your physical NICs. These names vary between systems.

```
# Run this command on the Proxmox host (login via SSH or console):
ip a

# Look for two interfaces:
# Built-in NIC (WAN): typically named eno1, ens3, enp2s0, or eth0
# USB Ethernet (LAN): typically named enxXXXXXXXXXXXX
# (starts with 'enx' followed by the MAC address of the USB adapter)

# Write down BOTH names exactly before editing the config file
# Using the wrong name will break network connectivity
```

5.3 Full /etc/network/interfaces Configuration (V2 — Three Bridges)

This is the complete network configuration file for the Proxmox host. Replace the NIC names with your actual interface names identified in step 5.2. Replace [WAN_SUBNET] and [LAN_SUBNET] with your actual subnet values.

```
# Edit the network configuration:
nano /etc/network/interfaces

# Replace the entire file content with the following:

auto lo
iface lo inet loopback

# == WAN Bridge - Built-in NIC ==
# Purpose: Proxmox management UI + pfSense WAN + Tailscale WAN fallback
auto vmbr0
iface vmbr0 inet static
    address [WAN_SUBNET].10/24
    gateway [WAN_SUBNET].1
    bridge-ports eno1                # <-- replace with your built-in NIC name
    bridge-stp off
    bridge-fd 0
    bridge-vlan-aware no

# == LAN Client Bridge - USB NIC ==
# Purpose: LAN client devices + Pi-hole + Wazuh + Tailscale LAN
# Suricata's input interface (exxxx8) also connects here
auto vmbr1
iface vmbr1 inet static
    address [LAN_SUBNET].100/24
    bridge-ports enxXXXXXXXXXXXX    # <-- replace with your USB NIC name
    bridge-stp off
```

```

bridge-fd 0
bridge-vlan-aware no

# == Internal Bridge - NO physical NIC ==
# Purpose: Suricata output (exxxxx9) to pfSense LAN (vtnet1)
# V2 ADDITION: Completes the inline inspection path
# This bridge has NO physical NIC - it is purely internal
auto vmbr2
iface vmbr2 inet manual
    bridge-ports none
    bridge-stp off
    bridge-fd 0
    bridge-vlan-aware no
# NOTE: vmbr2 has no IP address - it is a pure internal virtual switch
# Traffic flows: vmbr1 → Suricata VM (br0) → vmbr2 → pfSense LAN

```

5.4 Applying and Validating

```

# Apply the new network configuration
# WARNING: This may briefly drop network connectivity to Proxmox UI
ifreload -a

# If ifreload causes problems, a clean reboot is safer:
reboot

# After applying or rebooting, validate all three bridges exist:
brctl show
# Expected output: vmbr0, vmbr1, and vmbr2 all listed
# vmbr2 will show no ports listed - this is correct and expected

# Verify IP addresses are assigned correctly:
ip a
# Expected:
#   vmbr0 → [WAN_SUBNET].10/24
#   vmbr1 → [LAN_SUBNET].100/24
#   vmbr2 → no IP (correct - internal bridge has no IP)

# Verify WAN connectivity:
ping -c 4 [WAN_SUBNET].1
# Expected: 4 replies from your home router

# Verify Proxmox UI still accessible:
# https://[WAN_SUBNET].10:8006 should load normally

```

Chapter 6 — pfSense — Gateway & Firewall

CRITICAL pfSense is the most critical component in the entire system. Every other service depends on it for routing, DHCP, and DNS enforcement. Build and validate pfSense completely before creating any other VM or LXC.

6.1 What Is pfSense?

pfSense is a full-featured open-source firewall and router operating system based on FreeBSD. It runs as VM 100 and is responsible for four critical functions:

- **ROUTING:** Moves traffic between the WAN (internet) and LAN (home network)
- **DHCP SERVER:** Assigns IP addresses to all devices when they join the network
- **DNS ENFORCEMENT:** Forces all DNS queries through Pi-hole using firewall rules and NAT redirect
- **FIREWALL:** Allows or blocks traffic based on rules — including blocking DNS bypass attempts

NOTE V2 CHANGE: In V2, pfSense's LAN interface (vtnet1) connects to vmbr2 — NOT vmbr1. Traffic from LAN clients goes through vmbr1 → Suricata VM (br0) → vmbr2 → pfSense. This is the inline inspection path. pfSense never sees traffic that hasn't been inspected by Suricata first.

6.2 VM Creation in Proxmox

Create the pfSense VM using the Proxmox web UI. These settings are specific and must be followed exactly:

Proxmox Web UI ([https://\[WAN_SUBNET\].10:8006](https://[WAN_SUBNET].10:8006))
Click 'Create VM' in the top right corner

— GENERAL tab —

VM ID: 100
Name: pfSense

— OS tab —

ISO Image: pfSense-CE-[version]-amd64.iso
(Download from: <https://www.pfsense.org/download/>)
(Choose: AMD64, DVD Image, Netgate mirror)
Type: Other

— SYSTEM tab —

Machine: Default (i440fx)
BIOS: SeaBIOS

— DISKS tab —

Bus/Device: VirtIO Block
Disk size: 12 GiB
Storage: local-lvm

— CPU tab —

Cores: 4
Type: host

— MEMORY tab —

Memory: 4096 MiB (4 GB)

— NETWORK tab (this is the FIRST NIC — WAN) —

Bridge: vmbr0 <-- WAN interface
Model: VirtIO (paravirt)

Click 'Finish' to create the VM

— ADD SECOND NIC (LAN) — after VM is created —————

Proxmox → VM 100 → Hardware → Add → Network Device

Bridge: vmbr2 <-- LAN interface (V2: Suricata output side)

Model: VirtIO (paravirt)

Click Add

CRITICAL NIC ORDER IS CRITICAL: net0 must be vmbr0 (WAN). net1 must be vmbr2 (LAN). If these are swapped, your home network will be directly exposed to the internet. Double-check in Hardware tab before starting the VM.

6.3 pfSense Installation

Start VM 100 and click the Console tab in Proxmox to see the VM display:

Step	Action	Verify With
1	VM boots from ISO → pfSense boot menu appears → press Enter or wait for auto-boot	Boot menu appears with pfSense logo
2	Select 'Install pfSense' → Accept defaults → Auto (UFS) partitioning → Confirm	Installation progress shown, completes without error
3	'Should VLANs be set up now?' → type n → Enter	VLAN setup skipped
4	'Enter the WAN interface name' → type vtnet0 → Enter	WAN = vtnet0 confirmed
5	'Enter the LAN interface name' → type vtnet1 → Enter	LAN = vtnet1 confirmed (this is vmbr2 in V2)
6	'Do you want to proceed?' → type y → Enter	pfSense configures interfaces and starts services
7	pfSense menu appears showing WAN and LAN IP addresses	Menu shows LAN IP as 192.168.1.1 (default — will be changed)

6.4 LAN & DHCP Configuration

Configure the LAN IP address and DHCP server from the pfSense console menu (option 2):

```
# From pfSense console menu:
# Select option 2: Set interface(s) IP address
# Select 2 for LAN interface

Configure IPv4 address LAN interface via DHCP? n
Enter the new LAN IPv4 address: [LAN_SUBNET].1
Enter the new LAN IPv4 subnet bit count: 24
For a WAN, enter the new LAN IPv4 upstream gateway address: (press Enter — none for LAN)
Configure IPv6 address LAN interface via DHCP6? n
Enter the new LAN IPv6 address: (press Enter — skip IPv6)
Do you want to enable the DHCP server on LAN? y
Enter the start address of the IPv4 client address range: [LAN_SUBNET].10
Enter the end address of the IPv4 client address range: [LAN_SUBNET].99
Do you want to revert to HTTP as the webConfigurator protocol? n

# pfSense will apply changes and show the new LAN IP
# Web UI is now accessible at: http://[LAN_SUBNET].1
# Default credentials: admin / pfsense → CHANGE IMMEDIATELY
```

After accessing the web UI, configure DHCP DNS settings:

```
# pfSense Web UI: Services → DHCP Server → LAN
# Under 'Servers' section:
DNS Server 1: [LAN_SUBNET].4      (Pi-hole – this is the ONLY DNS server)
DNS Server 2: (leave EMPTY)      (no secondary DNS – prevents Pi-hole bypass)
# Save and Apply Changes
```

CRITICAL Never configure a secondary DNS server in DHCP settings. If a secondary is set and Pi-hole goes down, devices will fall back to the secondary and bypass all DNS filtering entirely. Pi-hole must be the only DNS server given to clients.

6.5 Firewall Rules for DNS Enforcement

These firewall rules ensure no device can use any DNS server other than Pi-hole. Configure under Firewall → Rules → LAN:

Order	Protocol	Source	Destination	Port	Action	Description
1	TCP/UDP	LAN net	[LAN_SUBNET].4	53	PASS	Allow DNS to Pi-hole only
2	TCP/UDP	LAN net	any	53	BLOCK	Block all other DNS queries
3	TCP/UDP	LAN net	any	853	BLOCK	Block DNS-over-TLS bypass attempts
4	TCP/UDP	LAN net	[LAN_SUBNET].7	443	PASS	Allow Wazuh dashboard access
5	any	LAN net	any	any	PASS	Default allow outbound (must be LAST)

Rules are evaluated top to bottom. The first matching rule wins. Rule 1 explicitly allows DNS to Pi-hole. Rule 2 blocks DNS to everything else. Rule 3 blocks DNS-over-TLS (port 853) which some apps use to bypass port-53 filtering. Rule 5 (default allow) must be at the bottom.

6.6 Remote Syslog to Wazuh

Configure pfSense to send its firewall logs to the Wazuh SIEM. Do this after Wazuh is deployed (Chapter 12).

```
# pfSense Web UI: Status → System Logs → Settings
# Scroll to 'Remote Logging Options'

Enable Remote Logging: checked (tick the checkbox)
Source Address: Default (any)
IP Protocol: IPv4
Remote log servers: [LAN_SUBNET].7
Remote Syslog Port: 514
Remote Syslog Protocol: UDP
Remote Syslog Contents:
  ✓ Firewall Events
  ✓ System Events
  ✓ DHCP service events (optional but useful)

Click Save
```

6.7 Validation

```
# From a LAN-connected device (phone or laptop on Wi-Fi):
ping 8.8.8.8
# Expected: replies – confirms internet routing works

ping [LAN_SUBNET].1
# Expected: replies – confirms pfSense LAN is reachable

# Browser test:
# Open http://[LAN_SUBNET].1
# Expected: pfSense web UI login page loads

# Check DHCP assignment on a device:
# Windows: ipconfig /all
# Linux/Mac: ip a
# Expected IP: [LAN_SUBNET].10-99
# Expected Gateway: [LAN_SUBNET].1
# Expected DNS: [LAN_SUBNET].4
```

Chapter 7 — Access Point Configuration

7.1 Why Access Point Mode Is Required

Consumer wireless routers have multiple built-in functions: routing, NAT, DHCP server, DNS server, and Wi-Fi access point. Using such a device in full router mode on your LAN creates serious problems:

- Double NAT: Two devices performing NAT causes connectivity issues, breaks certain applications, and complicates troubleshooting
- Rogue DHCP: A second DHCP server on the network causes IP address conflicts — devices receive incorrect IP addresses and lose connectivity
- DNS bypass: The router's built-in DNS server bypasses Pi-hole, allowing malware to communicate without interception

In Access Point (AP) mode, the wireless router acts only as a Wi-Fi radio and ethernet switch. All routing, NAT, DHCP, and DNS are handled exclusively by pfSense and Pi-hole. The AP simply extends the LAN wirelessly.

7.2 Access Point Configuration Steps

These steps configure a typical consumer router as a pure access point. The exact UI may differ slightly between models, but the principles are identical:

Step	Action	Why / Verify
1	Factory reset the device: hold the reset button for 10 seconds until all lights flash and the device reboots	Clean starting state prevents conflicts from previous configuration
2	Connect a laptop directly to one of the numbered LAN ports on the device (not the WAN/Internet port). Open browser and go to the device's default IP (usually 192.168.0.1 or 192.168.1.1)	Default admin interface loads
3	Login with default credentials (usually admin/admin or admin/password). Dismiss any setup wizard.	Web interface loads with all settings visible
4	Network → LAN Settings → Set IP address to [LAN_SUBNET].2, subnet mask 255.255.255.0. Save.	AP management IP moves to SOC subnet. Browser will disconnect — reconnect to http://[LAN_SUBNET].2
5	DHCP Server settings → Disable the DHCP server completely. Save.	pfSense is now the only DHCP server. Setting shows 'Disabled'.
6	Wireless → Basic Settings → Set SSID (Wi-Fi network name) of your choice. Save.	Wi-Fi network name is configured
7	Wireless → Security → Select WPA2-PSK (AES). Set a strong Wi-Fi password. Save.	WPA2 encryption enabled. WPA3 if available is also acceptable.
8	Physical wiring: connect an Ethernet cable from the USB Ethernet adapter on the Proxmox host to one of the NUMBERED LAN ports on the access point. Do NOT use the WAN/Internet port.	Wi-Fi clients now get IP addresses from pfSense DHCP ([LAN_SUBNET].10-99)

CRITICAL NEVER connect to the AP's WAN/Internet port. This port is designed to receive a connection FROM an upstream router. Connecting your LAN cable to it re-activates the AP's router mode, re-enables its DHCP server, and creates double NAT. Always use a numbered LAN port.

7.3 Validation

```
# Connect a phone or laptop to the Wi-Fi network

# On Windows: Settings → Wi-Fi → Properties for connected network
# On Linux: ip a (look at assigned IP)
# On Mac: System Preferences → Network → Wi-Fi → Advanced → TCP/IP

# Expected values:
# IP address: [LAN_SUBNET].10 through [LAN_SUBNET].99
# Subnet mask: 255.255.255.0
# Default gateway: [LAN_SUBNET].1      (pfSense)
# DNS server: [LAN_SUBNET].4          (Pi-hole)

# If DNS shows anything other than [LAN_SUBNET].4, the firewall
# rules in Chapter 6.5 need to be reviewed.
```

Chapter 8 — Pi-hole — DNS Filtering

8.1 What Is Pi-hole?

Pi-hole is a network-wide DNS sinkhole and ad blocker. It operates as the DNS server for the entire network — every DNS query from every device passes through it before being resolved.

When a device asks 'What is the IP address of ads.doubleclick.net?', Pi-hole checks its blocklist database. If the domain is known malicious, Pi-hole returns 0.0.0.0 (a non-routable address). The device receives no IP to connect to — the connection never happens. The malware, ad, or tracker is blocked at the DNS level before any connection is made.

This protection applies to every device on the network automatically — phones, smart TVs, IoT devices, gaming consoles — without installing any software on those devices.

PROTECTION SCOPE Pi-hole blocks: advertisements, tracking scripts, malware domains, phishing sites, cryptomining scripts, and command-and-control domains used by malware. The blocklists in this setup cover approximately 1.2 million known malicious or unwanted domains.

8.2 LXC Creation

Pi-hole runs as a lightweight LXC container, not a full VM. First, download the Debian 13 template in Proxmox:

```
Proxmox Web UI → local storage → CT Templates → Templates button
Search for: debian-13-standard
Select: debian-13-standard_13.x-x_amd64.tar.zst
Click Download and wait for completion
```

— Create the LXC Container —
Proxmox → Create CT

General tab:

```
CT ID: 102
Hostname: pihole
Password: (set a strong password)
Unprivileged container: CHECKED (important for security)
```

Template tab:

```
Storage: local
Template: debian-13-standard_13.x-x_amd64.tar.zst
```

Disks tab:

```
Storage: local-lvm
Disk size: 8 GiB
```

CPU tab:

```
Cores: 2
```

Memory tab:

```
Memory: 1024 MiB
Swap: 512 MiB
```

Network tab:

```
Name: eth0
Bridge: vmbr1
IPv4: Static
IPv4/CIDR: [LAN_SUBNET].4/24
Gateway: [LAN_SUBNET].1
```

DNS tab:

```
DNS domain: local
DNS servers: 1.1.1.1    (temporary – will use Pi-hole itself after setup)
```

Click Finish → Start container after creation: YES

8.3 Pi-hole Installation

Access the Pi-hole container console via Proxmox (click LXC 102 → Console):

```
# Update system packages first
apt update && apt upgrade -y

# Install curl if not present
apt install curl -y

# Run the Pi-hole automated installer
curl -sSL https://install.pi-hole.net | bash

# The installer will present a text-based UI. Select:
# ┌─ Network interface: eth0
# ┌─ Upstream DNS provider: Custom
# │   ┌─ Primary: 1.1.1.1    (Cloudflare)
# │   └─ Secondary: 8.8.8.8  (Google – fallback only)
# └─ Third party lists: Keep defaults checked
# └─ Install Admin Web Interface: Yes
# └─ Install lighttpd web server: Yes
# └─ Enable query logging: Yes

# After installation, set the admin dashboard password:
pihole -a -p
# Enter and confirm your chosen password

# Access admin dashboard at: http://[LAN_SUBNET].4/admin
# Login with the password you just set
```

8.4 Recommended Blocklists

The default Pi-hole blocklists are a starting point. These additional blocklists provide comprehensive protection against malware, phishing, and tracking. Add them via: Admin Dashboard → Group Management → Adlists → Add:

List Name	URL	Blocks ~	Covers
OISD Full	https://big.oisd.nl	250k	Ads, malware, phishing, ransomware C2
Steven Black	raw.githubusercontent.com/StevenBlack/hosts/master/hosts	180k	Ads, malware, phishing
HaGeZi Multi Pro	raw.githubusercontent.com/hagezi/dns-blocklists/main/adblock/pro.txt	700k	Ads, tracking, malware, phishing
URLhaus	urlhaus-filter.pages.dev/urlhaus-filter-hosts.txt	50k	Active malware distribution sites
Phishing Army	phishing.army/download/phishing_army_blocklist_extended.txt	30k	Phishing and credential harvesting sites

```
# After adding all blocklists, compile the Gravity database:
pihole -g
# This downloads and compiles all blocklists into Pi-hole's database
# May take 1-2 minutes

# Required whitelists (these must not be blocked):
pihole -w packages.wazuh.com          # Wazuh updates
pihole -w raw.githubusercontent.com # GitHub raw content
pihole -w login.tailscale.com         # Tailscale authentication
pihole -w controlplane.tailscale.com  # Tailscale control plane
pihole -w pool.ntp.org               # Network time sync

# Verify Pi-hole is running:
pihole status
# Expected: DNS service running, Blocking enabled, X domains in blocklist
```

8.5 Validation

```
# From the Pi-hole LXC console:

# Test 1: Legitimate domain resolves correctly
nslookup google.com [LAN_SUBNET].4
# Expected: Returns Google's real IP address

# Test 2: Known ad/malware domain is blocked
nslookup doubleclick.net [LAN_SUBNET].4
# Expected: Returns 0.0.0.0 (blocked)

# Test 3: Pi-hole status
pihole status
# Expected: [✓] DNS service is running
#           [✓] Pi-hole blocking is Enabled
#           Blocking X domains (should be 1M+)
```

Chapter 9 — DNS Enforcement

CRITICAL Without DNS enforcement, any device can bypass Pi-hole by directly specifying a DNS server (e.g. 8.8.8.8) in its network settings. Malware regularly does this to avoid DNS-based blocking. This chapter makes DNS enforcement absolute — every query is intercepted, regardless of device configuration.

9.1 The Problem — DNS Bypass

Pi-hole is configured as the DHCP-assigned DNS server. However, any device can override DHCP-assigned DNS by manually configuring a different DNS server in its network settings. Some malware does this automatically. Smart TVs and gaming consoles often have hardcoded DNS servers.

The solution is a pfSense NAT redirect rule that intercepts ALL DNS traffic on port 53 and redirects it to Pi-hole — regardless of what DNS server the device thinks it's using. The device sends a query to 8.8.8.8:53, pfSense silently redirects it to Pi-hole:53, and the device never knows the difference.

9.2 NAT DNS Redirect Rule

Configure in pfSense Web UI: Firewall → NAT → Port Forward → Add:

```
Interface: LAN
Address Family: IPv4
Protocol: TCP/UDP
Source: LAN net
Source Port Range: any
Destination: NOT [LAN_SUBNET].4      (any destination EXCEPT Pi-hole itself)
Destination Port Range: DNS (53)
Redirect target IP: Single host → [LAN_SUBNET].4
Redirect target port: DNS (53)
Description: Force all DNS through Pi-hole
```

Click Save → Apply Changes

This rule says: any DNS query from any LAN device going to anywhere except Pi-hole itself → silently redirect to Pi-hole. The device does not know its query was redirected.

9.3 Disable IPv6 — Close the Bypass Gap

IPv6 DNS (port 53 over IPv6) bypasses the IPv4 NAT redirect rule. Disable IPv6 entirely to close this gap:

```
# pfSense Web UI: System → Advanced → Networking tab
Prefer IPv4 over IPv6: CHECKED
Click Save
```

```
# pfSense Web UI: Interfaces → WAN
IPv6 Configuration Type: None
Click Save
```

```
# pfSense Web UI: Interfaces → LAN
IPv6 Configuration Type: None
Click Save → Apply Changes
```

9.4 Validation

```
# Test from any LAN device (phone, laptop):

# Test 1: Normal DNS still works through Pi-hole
nslookup google.com
# Expected: Resolves to Google's IP normally

# Test 2: DNS bypass attempt is intercepted
nslookup doubleclick.net 8.8.8.8
# Expected: Returns 0.0.0.0 (was redirected to Pi-hole despite specifying 8.8.8.8)
# If this returns a real IP, the NAT redirect rule is not working

# Test 3: All queries appear in Pi-hole
# Open Pi-hole dashboard: http://[LAN_SUBNET].4/admin
# Go to Query Log
# Expected: ALL DNS queries from ALL devices appear here

VALIDATION PASSED when: nslookup doubleclick.net 8.8.8.8 returns 0.0.0.0
This confirms no device on the network can bypass Pi-hole DNS filtering.
```

NOTE This is one of the most important validations in the entire build. When this test passes, every device on the network — including those with hardcoded DNS, including malware — has its DNS controlled by Pi-hole.

Chapter 10 — Tailscale — Secure Remote Access

10.1 What Is Tailscale?

Tailscale is a VPN service built on the WireGuard protocol. It creates a secure peer-to-peer mesh network between your devices. Unlike traditional VPNs, Tailscale requires no port forwarding, no static public IP, and no complex configuration.

Once Tailscale is installed on the SOC and on your phone or laptop, both devices appear on the same virtual network. You can securely access the Wazuh dashboard, pfSense UI, and Proxmox from anywhere in the world without exposing any ports to the internet.

SECURITY NOTE Tailscale uses end-to-end encryption between peers. Even Tailscale itself cannot see your traffic. The Tailscale coordination server only helps devices find each other — it never sees the actual data.

10.2 Architecture

The Tailscale LXC has two network interfaces:

- eth0 on vmbr1 ([LAN_SUBNET].5) — LAN side, allows routing to all SOC components
- eth1 on vmbr0 (DHCP from WAN) — WAN fallback for Tailscale coordination

Tailscale advertises the LAN subnet ([LAN_SUBNET].0/24) as a route. Remote devices that connect to your Tailscale network can reach any device on your LAN using their normal LAN IPs.

CRITICAL --accept-dns=false IS MANDATORY. Without this flag, Tailscale overrides Pi-hole as the DNS server for every active VPN session. All DNS filtering stops for you when connected remotely. This is the single most common misconfiguration and the most dangerous omission.

10.3 LXC Creation

Proxmox → Create CT

```
General: CT ID: 103 | Hostname: tailscale | Unprivileged: YES
Template: debian-13-standard
Disks: 8 GiB | local-lvm
CPU: 2 cores
Memory: 512 MiB (Tailscale is very lightweight)
Network (eth0): Bridge vmbr1 | Static IPv4: [LAN_SUBNET].5/24 | GW: [LAN_SUBNET].1
```

After creation — add second NIC:

```
CT 103 → Hardware → Add → Network Device
    eth1 | Bridge: vmbr0 | IPv4: DHCP
```

BEFORE starting the container — add TUN device support:

On the PROXMOX HOST (not inside the container):

```
nano /etc/pve/lxc/103.conf
# Add these two lines at the end of the file:
lxc.cgroup2.devices.allow: c 10:200 rwm
lxc.mount.entry: /dev/net/tun dev/net/tun none bind,create=file
# Save file
# Now start the container: pct start 103
```

10.4 Installation and Setup

```
# Inside Tailscale LXC (Proxmox → CT 103 → Console):
apt update && apt upgrade -y

# Install Tailscale
curl -fsSL https://tailscale.com/install.sh | sh

# Enable IP forwarding (required for subnet routing)
sysctl -w net.ipv4.ip_forward=1
echo 'net.ipv4.ip_forward=1' >> /etc/sysctl.conf
sysctl -p

# Start Tailscale and authenticate
tailscale up \
  --advertise-routes=[LAN_SUBNET].0/24 \
  --accept-dns=false

# Tailscale will output a URL – open it in a browser to authenticate
# After authentication, the device appears in your Tailscale admin console

# Approve routes in Tailscale admin console:
# Go to: https://login.tailscale.com/admin/machines
# Find your SOC device → click the three-dot menu → Edit route settings
# Enable the subnet route for [LAN_SUBNET].0/24
# This allows remote devices to reach your entire home network
```

10.5 Architecture Overview — Tailscale Auth Steps Omitted

Specific authentication tokens and Tailscale account details are not included in this public document for security reasons. The installation steps above cover the complete technical configuration. Refer to the official Tailscale documentation at tailscale.com/kb for account-specific authentication procedures.

10.6 Validation

```
# Inside Tailscale LXC:
tailscale status
# Expected: Device shows as connected with a Tailscale IP
#           Peers listed if other Tailscale devices are online

# From outside your home network (phone on mobile data with Tailscale app):
ping [LAN_SUBNET].1
# Expected: Replies from pfSense – you are on your home LAN remotely

# Access Wazuh dashboard remotely:
# https://[LAN_SUBNET].7
# Expected: Wazuh login page loads over Tailscale VPN
```

Chapter 11 — Suricata — Intrusion Detection System (Inline Bridge Mode)

NOTE V2 UPDATE: Suricata is now inline in the traffic path — full packet visibility confirmed. Every packet from every LAN device passes through Suricata before reaching pfSense. SYN packets observed. Alerts generated. This chapter documents the V2 architecture change and complete configuration.

11.1 What Is Suricata?

Suricata is an open-source high-performance network IDS/IPS/NSM engine. It monitors network traffic in real time and compares each packet against a ruleset of thousands of known attack signatures. When Suricata recognises an attack pattern — such as a port scan, brute force attempt, exploit payload, or malware command-and-control communication — it generates a structured alert in `/var/log/suricata/eve.json`.

The Wazuh agent running on the Suricata VM reads `eve.json` continuously and ships every alert to the SIEM. This creates the detection pipeline: network traffic → Suricata IDS → `eve.json` → Wazuh agent → Wazuh SIEM → alert + Telegram notification.

11.2 Why V1 Failed — Root Cause Analysis

In V1, Suricata was connected as a normal VM on `vmbr1`, alongside pfSense, Pi-hole, and Wazuh. The assumption was that Suricata would see all LAN traffic. This assumption was wrong.

V1 Architecture (BROKEN — passive mode):

```
vmbr1 (LAN bridge)
|
+— pfSense LAN (vtnet1) ← traffic goes directly here
|
+— Suricata VM (eth0)    ← sees only its OWN traffic
|
+— Pi-hole, Wazuh, etc.
```

ROOT CAUSE A Proxmox bridge (`vmbr`) is a standard Linux bridge. It is not a network hub — it does not broadcast all traffic to all ports. A Linux bridge forwards unicast traffic only to the specific MAC address it is destined for. Suricata, connected as a normal VM, only received traffic addressed to its own MAC address. When running `tcpdump` on V1 Suricata, the capture showed only ARP packets — no TCP SYN packets, no scan traffic, no actual LAN traffic. Suricata generated zero alerts because it saw zero traffic.

11.3 V2 Solution — Inline Bridge Architecture

The V2 fix places Suricata physically in the path between LAN clients and pfSense. All traffic must pass through Suricata's internal Linux bridge (`br0`) to reach pfSense. Suricata cannot be bypassed.

V2 Architecture (WORKING — inline mode):

```
LAN Clients (phones, laptops, Wi-Fi)
| [USB NIC → vmbr1]
v
Suricata VM
  exxxx8 — connected to vmbr1 (client-side INPUT)
  |
  | [br0 — Linux bridge INSIDE Suricata VM]
  | *** ALL PACKETS INSPECTED HERE ***
  |
  exxxx9 — connected to vmbr2 (pfSense-side OUTPUT)
  |
  v [vmbr2 — internal Proxmox bridge]
pfSense LAN (vtnet1) ← receives all inspected traffic
```

Result: Every packet from every LAN device passes through Suricata. Suricata acts as a TRANSPARENT BRIDGE — devices have no idea it exists. SYN packets confirmed visible after V2 implementation. Scan alerts generated successfully.

11.4 Step 1 — Verify vmbr2 Exists

vmbr2 must exist in Proxmox before creating the Suricata VM. It should have been created in Chapter 5. Verify:

```
# On Proxmox host:
brctl show
# Expected: vmbr2 listed in the output
# vmbr2 will show no bridge ports — this is correct (no physical NIC attached)

# If vmbr2 is missing, add it to /etc/network/interfaces (Chapter 5.3) and run:
ifreload -a
```

11.5 Step 2 — Move pfSense LAN Interface to vmbr2

pfSense's LAN interface must be moved from vmbr1 to vmbr2 before creating the Suricata VM. This ensures pfSense receives traffic from Suricata's output, not directly from LAN clients.

```
Proxmox Web UI → VM 100 (pfSense) → Hardware tab
Find: Network Device (net1)
  It currently shows Bridge: vmbr1
Click Edit → Change Bridge to: vmbr2
Click OK
```

```
# Restart pfSense to apply the change:
qm reboot 100
# Wait ~30 seconds for pfSense to fully restart
```

CRITICAL Change pfSense LAN to vmbr2 BEFORE creating the Suricata VM. If you create Suricata first, there will be a window where pfSense has no LAN connection and all network access is lost. Order matters.

11.6 Step 3 — Create Suricata VM

```
Proxmox Web UI → Create VM

General:
  VM ID: 101
  Name: suricata

OS:
  ISO: Ubuntu Server 22.04 LTS
  (Download from: https://ubuntu.com/download/server)
  Type: Linux
  Kernel Version: 5.x or later

System:
  Machine: q35      (required — better VirtIO support)
  BIOS: SeaBIOS

Disks:
  Bus: VirtIO Block
  Disk size: 20 GiB
  Storage: local-lvm

CPU:
  Cores: 2
  Type: host

Memory:
  Memory: 4096 MiB (4 GB)

Network (FIRST NIC — client-side input):
  Bridge: vmbr1
  Model: VirtIO
```

Click Finish to create VM

— ADD SECOND NIC (pfSense-side output) —————
VM 101 → Hardware → Add → Network Device
Bridge: vmbr2 ← pfSense-side output
Model: VirtIO
Click Add

Start VM 101 → Install Ubuntu Server 22.04

IMPORTANT: During Ubuntu installation:

- Do NOT configure a static IP for exxxxxx8 or exxxxxx9
- The management IP will be assigned to br0 (the bridge) after install
- Temporarily use DHCP during install for internet access if needed

11.7 Step 4 — Create Linux Bridge Inside Suricata VM

After Ubuntu is installed, SSH in or use the Proxmox console. Run these commands immediately — they create the transparent bridge that makes Suricata inline:

```
# Step 1: Install required utilities
apt update && apt install -y bridge-utils ethtool

# Step 2: Create the bridge interface
ip link add name br0 type bridge

# Step 3: Add both NICs to the bridge
ip link set exxxxxx8 master br0
ip link set exxxxxx9 master br0

# Step 4: Bring all interfaces up
ip link set exxxxxx8 up
ip link set exxxxxx9 up
ip link set br0 up

# Step 5: Remove IP addresses from physical NICs (they should have no IP)
ip addr flush dev exxxxxx8
ip addr flush dev exxxxxx9

# Step 6: Assign the management IP to br0
ip addr add [LAN_SUBNET].6/24 dev br0
ip route add default via [LAN_SUBNET].1

# Verify bridge is correctly configured:
ip a show br0
# Expected: inet [LAN_SUBNET].6/24 shown on br0

brctl show
# Expected: br0 listed with exxxxxx8 and exxxxxx9 as member ports

# Test connectivity:
ping [LAN_SUBNET].1
# Expected: Replies from pfSense — traffic flows through bridge
ping 8.8.8.8
# Expected: Replies — internet works through the bridge
```

NOTE Your Ubuntu VM may use different NIC names. Run 'ip a' immediately after Ubuntu install to find your actual names. Common alternatives: ens18/ens19, eth0/eth1, or enp3s0/enp3s1. Substitute your actual names everywhere exxxxxx8 and exxxxxx9 appear in this chapter.

11.8 Step 5 — Netplan Persistent Configuration

The commands in Step 4 are temporary — they are lost on reboot. This Netplan configuration makes the bridge permanent. It survives reboots and is the authoritative configuration:

```
# Edit or create the Netplan config file:
nano /etc/netplan/00-installer-config.yaml

# Replace entire file content with:

network:
  version: 2
  renderer: networkd
  ethernets:
    exxxx8:
      dhcp4: no
    exxxx9:
      dhcp4: no
  bridges:
    br0:
      interfaces:
        - exxxx8
        - exxxx9
      addresses:
        - [LAN_SUBNET].6/24
      routes:
        - to: default
          via: [LAN_SUBNET].1
      nameservers:
        addresses:
          - [LAN_SUBNET].4
      parameters:
        stp: false
        forward-delay: 0
        dhcp4: no

# Apply the configuration:
netplan apply

# Verify after apply:
ip a show br0
# Expected: [LAN_SUBNET].6/24 on br0

ping [LAN_SUBNET].1    # pfSense reachable
ping 8.8.8.8           # internet works through bridge

# Test persistence — reboot and verify config survives:
reboot
# After reboot:
ip a show br0          # Should still show [LAN_SUBNET].6/24
ping [LAN_SUBNET].1    # Should still work
```

11.9 Step 6 — Disable NIC Hardware Offloading

Hardware offloading features (GRO, LRO, TSO, GSO) are performance optimisations that batch and reassemble packets at the hardware level before delivering them to software. This is helpful for normal traffic but breaks Suricata's inspection — Suricata sees modified, reassembled packets rather than original network frames, causing it to miss attacks and fail signature matching.

These features **MUST** be permanently disabled on both bridge member interfaces:

Feature	What It Does	Impact If Enabled on Suricata Bridge
GRO (Generic Receive Offload)	Merges multiple small packets into one large packet on receive	Suricata sees merged packets, missing individual connection details and flags
LRO (Large Receive Offload)	Hardware-level packet merging, more aggressive than GRO	Suricata cannot inspect original packet headers — detection accuracy severely reduced
TSO (TCP Segmentation Offload)	Lets the NIC handle TCP segmentation instead of the OS	Suricata sees oversized frames that violate TCP spec — rules may not match
GSO (Generic Segmentation Offload)	Software-level segmentation deferral	Suricata receives packets that don't reflect actual wire traffic

```
# Disable offloading immediately (temporary – for testing):
ethtool -K xxxxxx8 gro off lro off tso off gso off
ethtool -K xxxxxx9 gro off lro off tso off gso off

# Verify offloading is disabled:
ethtool -k xxxxxx8 | grep -E 'generic-receive|large-receive|tcp-seg|generic-seg'
# Expected: all show 'off'

# Make permanent using a systemd service:
cat > /etc/systemd/system/disable-offload.service << 'EOF'
[Unit]
Description=Disable NIC offloading for Suricata inline bridge
After=network.target

[Service]
Type=oneshot
ExecStart=/sbin/ethtool -K xxxxxx8 gro off lro off tso off gso off
ExecStart=/sbin/ethtool -K xxxxxx9 gro off lro off tso off gso off
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable disable-offload.service
systemctl start disable-offload.service

# Verify service is active:
systemctl status disable-offload.service
# Expected: active (exited) – this is correct for a oneshot service
```

11.10 Step 7 — Install Suricata

```
# Install Suricata and the rule management tool:
apt install -y suricata suricata-update

# Verify installation:
suricata --build-info | head -5
# Expected: Shows Suricata version, build information

# Download the Emerging Threats Open ruleset (free, updated daily):
sudo suricata-update
# Downloads to: /var/lib/suricata/rules/suricata.rules
# This may take a few minutes – downloading tens of thousands of rules
```

```
# Set up automatic weekly rule updates via cron:
crontab -e
# Add this line:
0 3 * * 1 /usr/bin/suricata-update && systemctl restart suricata
# This runs every Monday at 3am - updates rules and restarts Suricata
```

11.11 Step 8 — Configure suricata.yaml

The main Suricata configuration file requires changes for the inline bridge mode. Only the sections that need modification are shown:

```
sudo nano /etc/suricata/suricata.yaml

# — Section 1: HOME_NET —————
# Find 'vars:' section near top of file
vars:
  address-groups:
    HOME_NET: '[LAN_SUBNET].0/24'
    EXTERNAL_NET: '!$HOME_NET'

# — Section 2: af-packet - CRITICAL: BOTH interfaces —————
# Find the 'af-packet:' section
# V2 CHANGE: Must monitor BOTH exxxxxx8 AND exxxxxx9
# This captures all traffic in both directions through the bridge
af-packet:
  - interface: exxxxxx8
    cluster-id: 99
    cluster-type: cluster_flow
    defrag: yes
    use-mmap: yes
    tpacket-v3: yes
  - interface: exxxxxx9
    cluster-id: 98
    cluster-type: cluster_flow
    defrag: yes
    use-mmap: yes
    tpacket-v3: yes

# — Section 3: EVE JSON output —————
# Find 'outputs:' section
outputs:
  - eve-log:
      enabled: yes
      filetype: regular
      filename: /var/log/suricata/eve.json
      types:
        - alert:
            payload: yes
            payload-printable: yes
            metadata: yes
        - dns
        - http:
            extended: yes
        - tls:
            extended: yes
        - flow
        - stats:
            totals: yes

# — Section 4: Rule files —————
rule-files:
  - suricata.rules

# — Section 5: Community ID (enables event correlation) —————
community-id: true
```

```
# — Test configuration before starting —————
sudo suricata -T -c /etc/suricata/suricata.yaml -v
# Expected: 'Configuration provided was successfully loaded.'
# If errors appear, fix them before proceeding

# Enable and start Suricata:
sudo systemctl enable suricata
sudo systemctl start suricata

# Verify running:
sudo systemctl status suricata
# Expected: Active: active (running)
```

11.12 Step 9 — Install Wazuh Agent on Suricata VM

The Wazuh agent reads eve.json and ships Suricata's alerts to the SIEM:

```
# Add Wazuh repository:
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --dearmor \
  -o /usr/share/keyrings/wazuh.gpg
echo 'deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
  https://packages.wazuh.com/4.x/apt/ stable main' \
  | tee /etc/apt/sources.list.d/wazuh.list
apt update

# Install agent (pointing to Wazuh Manager):
WAZUH_MANAGER='[LAN_SUBNET].7' apt install wazuh-agent -y

# Configure agent to read eve.json:
nano /var/ossec/etc/ossec.conf
# Add inside the <ossec_config> block:
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>

# Enable and start agent:
systemctl daemon-reload
systemctl enable wazuh-agent
systemctl start wazuh-agent
```

11.13 Validation & Testing — All Tests Must Pass

```
# — Test 1: Verify bridge is correctly configured —————
brctl show
# Expected: br0 listed with exxxxx8 and exxxxx9 as ports

ip a show br0
# Expected: [LAN_SUBNET].6/24 assigned to br0

ping [LAN_SUBNET].1 # pfSense reachable through bridge
ping 8.8.8.8        # internet works through bridge

# — Test 2: Verify Suricata sees real traffic —————
# Start packet capture on Suricata's input interface:
sudo tcpdump -i exxxxx8 -n tcp
# On ANOTHER LAN device: open any website
# Expected: SYN packets (Flags [S]) visible in tcpdump output
# If only ARP packets appear: the inline bridge is not correctly configured

# — Test 3: Generate a scan and verify Suricata alert —————
# From another LAN device (Windows: open Command Prompt):
```

```
nmap -sS [LAN_SUBNET].7

# On Suricata VM – check for alert:
sudo tail -20 /var/log/suricata/fast.log
# Expected: ET SCAN or similar alert with source IP

sudo tail -5 /var/log/suricata/eve.json | python3 -m json.tool
# Expected: JSON alert with event_type: alert

# — Test 4: Verify alert reaches Wazuh SIEM —————
# On Wazuh VM:
/var/ossec/bin/agent_control -l
# Expected: Suricata agent shows Status: Active

# Wazuh Dashboard → Security Events
# Filter by agent: suricata
# Expected: IDS alerts visible in the SIEM dashboard
```

NOTE CONFIRMED WORKING in V2: After inline bridge configuration — SYN packets visible in tcpdump. nmap port scan generates ET SCAN alerts in fast.log and eve.json. Wazuh agent reads eve.json and alerts appear in SIEM. Configuration persists after reboot (Netplan + systemd offload service).

Chapter 12 — Wazuh — SIEM Platform

Wazuh is the central intelligence system of the Home SOC. It collects logs from every component, applies detection rules, generates alerts, and provides a web-based dashboard for investigation. Without Wazuh, each component operates in isolation with no correlation or centralised visibility.

Component	Role	What It Does
Wazuh Manager	Log processing engine	Receives all logs, decodes them, applies detection rules, generates alerts
Wazuh Indexer	Log storage	OpenSearch-based database storing all events for search and historical analysis
Wazuh Dashboard	Visualisation UI	Web-based interface for alert investigation, agent management, and dashboards

12.1 VM Creation

Wazuh is resource-intensive — it requires more CPU and RAM than any other component. Do not reduce these specifications:

Proxmox Web UI → Create VM

General:

VM ID: 104
Name: wazuh

OS:

ISO: Ubuntu Server 22.04 LTS
Type: Linux

System:

Machine: q35
BIOS: SeaBIOS

Disks:

Bus: VirtIO Block
Disk size: 80 GiB (Wazuh index data grows quickly)
Storage: local-lvm

CPU:

Cores: 6 (Wazuh indexer needs multiple cores)
Type: host

Memory:

Memory: 8192 MiB (8 GB MINIMUM for Wazuh 4.14.x)
(Wazuh indexer JVM alone uses 4-6 GB)

Network:

Bridge: vmbr1
Model: VirtIO

Install Ubuntu Server 22.04 normally

During install: set hostname 'wazuh', create a user

Do NOT configure static IP during install — do it after via Netplan

After Ubuntu install, set static IP:

```
nano /etc/netplan/00-installer-config.yaml
```

```
network:
```

```
  version: 2
```

```

ethernets:
  ens18:
    addresses:
      - [LAN_SUBNET].7/24
    routes:
      - to: default
        via: [LAN_SUBNET].1
    nameservers:
      addresses:
        - [LAN_SUBNET].4
    dhcp4: no

netplan apply
ping [LAN_SUBNET].1 # verify connectivity

```

12.2 Wazuh 4.14.2 Installation

Wazuh provides an all-in-one installer that sets up all three components (Manager, Indexer, Dashboard) on a single machine:

```

# Add Wazuh repository GPG key:
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --dearmor \
  -o /usr/share/keyrings/wazuh.gpg

# Add Wazuh repository:
echo 'deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
  https://packages.wazuh.com/4.x/apt/ stable main' \
  | tee /etc/apt/sources.list.d/wazuh.list

apt update

# Download and run the all-in-one installer:
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
bash wazuh-install.sh -a

# Installation takes 5-10 minutes
# At the end, the installer outputs admin credentials:
# SAVE THESE IMMEDIATELY – they cannot be recovered without resetting

# Example output:
# INFO: --- Summary ---
# INFO: You can access the web interface https://[LAN_SUBNET].7
#       User: admin
#       Password: <generated-password>

# Change admin password after first login:
# /usr/share/wazuh-indexer/plugins/opensearch-security/tools/wazuh-passwords-tool.sh \
#   -u admin -p YourNewStrongPassword

```

12.3 External HDD Setup for Log Storage

The Wazuh indexer generates significant data over time. Moving the index to an external USB HDD prevents the VM's internal disk from filling up and silently killing the SIEM:

```

# Attach USB HDD: Proxmox → VM 104 → Hardware → Add → USB Device
# Select your external HDD from the dropdown

# Inside Wazuh VM – identify the HDD:
lsblk
# Look for a disk without partitions – likely sdb or sdc
# Verify it is the correct disk before formatting

```

```

# Format the HDD (WARNING: this permanently erases all data on it):
mkfs.ext4 /dev/sdb

# Create mount point and mount:
mkdir -p /mnt/hdd
mount /dev/sdb /mnt/hdd

# Get UUID for permanent mount (survives reboots):
blkid /dev/sdb
# Note the UUID value (format: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx)

# Add to fstab for automatic mounting on boot:
nano /etc/fstab
# Add this line (replace UUID with your actual UUID):
UUID=your-actual-uuid /mnt/hdd ext4 defaults,nofail 0 2

# Test fstab entry without rebooting:
mount -a
# If no errors: fstab is correct

# Stop Wazuh services before moving data:
systemctl stop wazuh-manager wazuh-indexer wazuh-dashboard

# Create directories on HDD:
mkdir -p /mnt/hdd/wazuh-indexer
mkdir -p /mnt/hdd/wazuh-logs

# Move existing index data to HDD:
rsync -av /var/lib/wazuh-indexer/ /mnt/hdd/wazuh-indexer/

# Update Wazuh indexer to use HDD path:
nano /etc/wazuh-indexer/opensearch.yml
# Find and change these lines:
path.data: /mnt/hdd/wazuh-indexer
path.logs: /mnt/hdd/wazuh-logs

# Restart all Wazuh services:
systemctl start wazuh-indexer wazuh-manager wazuh-dashboard

# Verify all three are running:
systemctl status wazuh-indexer wazuh-manager wazuh-dashboard
# All three should show: Active: active (running)

```

12.4 Validation

```

# Verify all three Wazuh services are running:
systemctl status wazuh-indexer wazuh-manager wazuh-dashboard
# Expected: all three showing 'active (running)'

# Verify external HDD is mounted and being used:
df -h /mnt/hdd
# Expected: shows disk usage for the HDD

# List connected agents (will be empty until agents are installed):
/var/ossec/bin/agent_control -l

# Access dashboard:
# https://[LAN_SUBNET].7
# Login with admin credentials saved during installation
# Expected: Wazuh dashboard loads with Overview page

```

Chapter 13 — Log Integration Pipeline

CRITICAL This chapter activates the SIEM. Until all log integrations are complete, Wazuh is installed but completely blind — it receives no data and generates no alerts. Follow each section in order.

After completing this chapter, Wazuh will receive real-time events from: pfSense firewall, Pi-hole DNS, Suricata IDS, Tailscale VPN, Proxmox hypervisor, and Windows endpoints. All events appear in a single dashboard for correlation and investigation.

13.1 pfSense → Wazuh (Syslog UDP 514)

pfSense sends firewall logs via syslog. This integration gives Wazuh visibility into every firewall allow/block decision, DHCP lease assignment, and new device joining the network.

Step 1 — Configure pfSense to send logs:

```
# pfSense Web UI: Status → System Logs → Settings
# Scroll to 'Remote Logging Options'

Enable Remote Logging: CHECKED
Remote log servers: [LAN_SUBNET].7
Remote Syslog Port: 514
Remote Syslog Protocol: UDP
Remote Syslog Contents:
  ✓ Firewall Events
  ✓ System Events
  ✓ DHCP service events

Click Save
```

Step 2 — Configure Wazuh Manager to receive syslog:

```
# On Wazuh VM:
nano /var/ossec/etc/ossec.conf

# Add inside <ossec_config> block:
<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>[LAN_SUBNET].1</allowed-ips>
</remote>

# Restart Wazuh Manager:
systemctl restart wazuh-manager

# Verify Wazuh is listening on UDP 514:
netstat -ulnp | grep 514
# Expected: line showing 0.0.0.0:514 LISTEN

# Test: browse a website from a LAN device, then check Wazuh:
# Wazuh Dashboard → Discover → search agent.name:pfSense
# Expected: filterlog entries appearing
```

13.2 Pi-hole → Wazuh (Wazuh Agent)

The Pi-hole agent ships DNS query logs to Wazuh. This enables detection of: suspicious domain queries, DNS-based malware communication, and abnormal query patterns from any device.

```
# Inside Pi-hole LXC – access via Proxmox console (pct enter 102):

# Install prerequisites:
apt install curl gnupg -y

# Add Wazuh repository:
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --dearmor \
  -o /usr/share/keyrings/wazuh.gpg
echo 'deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
  https://packages.wazuh.com/4.x/apt/ stable main' \
  | tee /etc/apt/sources.list.d/wazuh.list
apt update

# Install Wazuh agent pointing to Wazuh Manager:
WAZUH_MANAGER='[LAN_SUBNET].7' apt install wazuh-agent -y

# Configure agent to read Pi-hole logs:
nano /var/ossec/etc/ossec.conf
# Add inside <ossec_config>:
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/pihole/pihole.log</location>
</localfile>

# Enable and start agent:
systemctl daemon-reload
systemctl enable wazuh-agent
systemctl start wazuh-agent

# Verify agent is running:
systemctl status wazuh-agent
# Expected: active (running)
```

13.3 Suricata → Wazuh (Agent + eve.json)

The Suricata agent was installed in Chapter 11. Verify the configuration is correct:

```
# On Suricata VM – verify ossec.conf has eve.json configured:
grep -A4 'eve' /var/ossec/etc/ossec.conf

# Expected output:
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>

# If missing, add it and restart:
nano /var/ossec/etc/ossec.conf
# Add inside <ossec_config>
systemctl restart wazuh-agent

# Test: generate a port scan from another LAN device:
nmap -sS [LAN_SUBNET].7

# Verify alert reaches Wazuh:
# Wazuh Dashboard → Security Events → filter agent.name: suricata
# Expected: IDS alert appears within 30 seconds
```

13.4 Tailscale → Wazuh (Agent + journald)

The Tailscale agent ships VPN connection events. This gives Wazuh visibility into: remote access sessions, new device connections via VPN, and Tailscale peer status changes.

```
# Inside Tailscale LXC (pct enter 103):

# Install Wazuh agent:
WAZUH_MANAGER='[LAN_SUBNET].7' apt install wazuh-agent -y

# Configure agent to read Tailscale journald logs:
nano /var/ossec/etc/ossec.conf
# Add inside <ossec_config>:
<localfile>
  <log_format>journald</log_format>
  <location>journald</location>
  <filter>_SYSTEMD_UNIT=tailscaled.service</filter>
</localfile>

# Enable and start agent:
systemctl daemon-reload
systemctl enable wazuh-agent
systemctl start wazuh-agent
```

13.5 Proxmox → Wazuh (Agent + syslog)

The Proxmox agent ships hypervisor system events and web UI access logs. This gives Wazuh visibility into: VM state changes, Proxmox UI logins, and system-level events on the host.

```
# On the Proxmox HOST (not inside any VM):

# Install Wazuh agent:
WAZUH_MANAGER='[LAN_SUBNET].7' apt install wazuh-agent -y

# Configure agent to read Proxmox logs:
nano /var/ossec/etc/ossec.conf
# Add inside <ossec_config>:
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/syslog</location>
</localfile>
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/pveproxy/access.log</location>
</localfile>

# Enable and start agent:
systemctl daemon-reload
systemctl enable wazuh-agent
systemctl start wazuh-agent
```

13.6 Windows Endpoint — Wazuh Agent + Sysmon

The Windows agent combined with Sysmon provides detailed endpoint telemetry: process creation, network connections, file modifications, registry changes, and driver loads. This is essential for detecting living-off-the-land attacks that leave no network footprint.

```
# Step 1: Install Wazuh Agent on Windows
# Wazuh Dashboard → Agents → Deploy New Agent
# Select: Windows → enter Manager IP: [LAN_SUBNET].7
# Download the MSI installer
# Run MSI as Administrator on the Windows machine
```

```
# Accept defaults – agent will auto-connect to Wazuh Manager

# Verify agent is running:
# Windows Services (services.msc) → look for 'Wazuh'
# Or PowerShell: Get-Service WazuhSvc
# Expected: Status Running

# Step 2: Install Sysmon for detailed endpoint logging
# Download Sysmon from:
# https://docs.microsoft.com/en-us/sysinternals/downloads/sysmon

# Download SwiftOnSecurity Sysmon config (recommended baseline):
# https://github.com/SwiftOnSecurity/sysmon-config
# Download: sysmonconfig-export.xml

# Install Sysmon with the config:
# Open Command Prompt as Administrator:
sysmon64.exe -accepteula -i sysmonconfig-export.xml
# Sysmon is now running and logging detailed endpoint events

# Step 3: Configure Wazuh agent to collect Sysmon events
# Edit: C:\Program Files (x86)\ossec-agent\ossec.conf
# Add inside <ossec_config>:
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>

# Step 4: Restart Wazuh agent to apply config change
# Command Prompt as Administrator:
net stop WazuhSvc
net start WazuhSvc
```

13.7 End-to-End Validation

After completing all integrations, verify every source is active and delivering logs:

Source	Validation Command / Method	Expected Result
All agents	<code>/var/ossec/bin/agent_control -l</code> (on Wazuh VM)	All agents show Status: Active
pfSense logs	Wazuh Dashboard → Discover → search agent.name:pfSense	filterlog entries visible in real time
Pi-hole logs	Wazuh Dashboard → Discover → search agent.name:pihole	DNS query entries appearing
Suricata alerts	Run <code>nmap -sS [LAN_SUBNET].7</code> from a LAN device	IDS alert appears in Wazuh Security Events within 60s
Windows endpoint	Wazuh Dashboard → Agents → look for Windows agent	Windows agent shows Active, Sysmon events appearing
Disk health	<code>df -h /mnt/hdd</code> (on Wazuh VM)	Less than 70% used

Chapter 14 — Wazuh Detection Rules

Custom Wazuh rules translate raw log data into meaningful security alerts with context and severity levels. Without custom rules, many important security events generate no alert. This chapter provides the complete custom decoder and rule set for this Home SOC.

14.1 How Rules Work

Wazuh processes every incoming log entry through two stages:

- **DECODER:** Extracts structured fields from raw text (e.g. extracts source IP from a firewall log line)
- **RULE:** Matches patterns in decoded fields and generates an alert with a severity level (1-15)

Severity levels: 1-6 = informational, 7-11 = warning/suspicious, 12-15 = critical. The Telegram integration triggers on level 7+.

File locations:

```
Custom decoders: /var/ossec/etc/decoders/local_decoder.xml
Custom rules:    /var/ossec/etc/rules/local_rules.xml
```

After any change to either file:

```
systemctl restart wazuh-manager
```

Test rules without restarting:

```
/var/ossec/bin/wazuh-logtest
(paste a log line and see which rules match)
```

14.2 Complete Custom Decoder File

Add all these decoders to `/var/ossec/etc/decoders/local_decoder.xml`:

```
<!-- pfSense firewall log decoder -->
<decoder name='pfsense_firewall'>
  <prematch>filterlog:</prematch>
</decoder>

<!-- pfSense DHCP event decoder -->
<decoder name='pfsense_dhcp'>
  <parent>pfsense_firewall</parent>
  <prematch>DHCPACK</prematch>
  <regex>DHCPACK on (\S+) to (\S+) via</regex>
  <order>srcip,id</order>
</decoder>

<!-- pfSense DHCP release/expire -->
<decoder name='pfsense_dhcp_leave'>
  <parent>pfsense_firewall</parent>
  <prematch>DHCPLEASE|DHCPRELEASE</prematch>
</decoder>

<!-- Pi-hole DNS base decoder -->
<decoder name='pihole_dns'>
  <prematch>dnsmasq</prematch>
</decoder>

<!-- Pi-hole blocked domain decoder -->
<decoder name='pihole_blocked'>
  <parent>pihole_dns</parent>
  <prematch>gravity blocked</prematch>
  <regex>gravity blocked (\S+) is</regex>
  <order>url</order>
</decoder>
```

```
<!-- Tailscale VPN decoder -->
<decoder name='tailscale_vpn'>
  <prematch>tailscaled</prematch>
</decoder>
```

14.3 Complete Custom Rules File

Add all these rules to `/var/ossec/etc/rules/local_rules.xml`:

```
<!-- ===== pfSense / DHCP Rules ===== -->
<group name='pfsense,dhcp,network,local'>

  <!-- New device joins network - DHCP lease granted -->
  <rule id='100900' level='5'>
    <decoded_as>pfsense_dhcp</decoded_as>
    <description>New device joined network: $(srcip)</description>
    <group>authentication_success,pci_dss_10.2.5</group>
  </rule>

  <!-- Device leaves network - DHCP release or expiry -->
  <rule id='100901' level='3'>
    <decoded_as>pfsense_dhcp_leave</decoded_as>
    <description>Device left network (DHCP release or expiry)</description>
  </rule>

</group>

<!-- ===== Pi-hole / DNS Rules ===== -->
<group name='pihole,dns,threat,local'>

  <!-- Single malicious domain blocked -->
  <rule id='100601' level='10'>
    <decoded_as>pihole_blocked</decoded_as>
    <description>Pi-hole blocked malicious domain: $(url)</description>
    <group>attack,pci_dss_10.6.1</group>
  </rule>

  <!-- Repeated blocks - possible malware C2 beaoning -->
  <!-- Triggers when 5 blocks happen within 120 seconds -->
  <rule id='100602' level='12' frequency='5' timeframe='120'>
    <if_matched_sid>100601</if_matched_sid>
    <description>Repeated DNS block - possible malware C2 beaoning from $(srcip)</description>
    <group>attack,malware,pci_dss_10.6.1</group>
  </rule>

</group>

<!-- ===== Suricata IDS Rules ===== -->
<group name='suricata,ids,local'>

  <!-- Port scan or reconnaissance detected -->
  <rule id='100801' level='10'>
    <decoded_as>json</decoded_as>
    <field name='event_type'>alert</field>
    <field name='alert.category'>Attempted Information Leak</field>
    <description>Suricata: Port scan or network reconnaissance detected from
$(src_ip)</description>
    <group>attack,pci_dss_11.4</group>
  </rule>

  <!-- Critical severity IDS alert (Suricata severity 1) -->
  <rule id='100802' level='12'>
    <decoded_as>json</decoded_as>
    <field name='event_type'>alert</field>
```

```

    <field name='alert.severity'>1</field>
    <description>Suricata: Critical severity IDS alert - $(alert.signature)</description>
    <group>attack,ids_alert,pci_dss_11.4</group>
</rule>

<!-- High severity IDS alert (Suricata severity 2) -->
<rule id='100803' level='10'>
    <decoded_as>json</decoded_as>
    <field name='event_type'>alert</field>
    <field name='alert.severity'>2</field>
    <description>Suricata: High severity IDS alert - $(alert.signature)</description>
    <group>attack,ids_alert</group>
</rule>

</group>

<!-- ===== Tailscale VPN Rules ===== -->
<group name='tailscale,vpn,local'>

    <!-- New VPN connection or peer -->
    <rule id='100701' level='8'>
        <decoded_as>tailscale_vpn</decoded_as>
        <match>node discovered|new peer|WireGuard handshake</match>
        <description>New device connected via Tailscale VPN</description>
        <group>authentication_success</group>
    </rule>

</group>

<!-- ===== Windows Endpoint Rules ===== -->
<group name='windows,endpoint,local'>

    <!-- Multiple failed Windows login attempts - brute force indicator -->
    <rule id='100501' level='10' frequency='5' timeframe='60'>
        <if_matched_sid>60204</if_matched_sid>
        <description>Multiple Windows login failures - possible brute force from
$(srcip)</description>
        <group>attack,authentication_failures</group>
    </rule>

</group>

<!-- Apply rules: systemctl restart wazuh-manager -->
<!-- Test: /var/ossec/bin/wazuh-logtest -->

```

14.4 Testing Rules with wazuh-logtest

After restarting wazuh-manager, test rules by feeding sample log lines:

```

# Run the log tester:
/var/ossec/bin/wazuh-logtest

# Paste a sample Pi-hole log line:
# Dec 25 10:30:15 dnsmasq[123]: gravity blocked evil-malware.com is 0.0.0.0

# Expected output shows:
# - Which decoder matched
# - Which rule fired
# - Alert level generated
# - Alert description

# Press Ctrl+C to exit when done

```

Chapter 15 — Storage Architecture

Wazuh generates significant log data over time. Understanding and monitoring storage prevents silent SIEM failures — when Wazuh runs out of disk space, it stops indexing without any visible error message.

Storage Type	What It Stores	Risk If Full
local (pve-root)	Proxmox ISO images, backups, templates, container rootfs	Proxmox backups fail silently. Templates cannot be downloaded.
local-lvm	VM virtual disks, LXC disk images	VMs freeze or crash with I/O errors. Data corruption risk.
external /mnt/hdd	Wazuh OpenSearch index data and logs	SIEM stops indexing new events. No alerts generated. No visible error on dashboard.

Storage	Warning Threshold	Critical Threshold	Action Required
local (pve-root)	70%	85%	Delete old ISO files and unused templates. Move backups to external.
local-lvm	75%	90%	Delete old VM/LXC snapshots immediately. Each snapshot consumes significant space.
external /mnt/hdd	80%	95%	Delete oldest Wazuh indices via Dashboard → Stack Management → Index Management.

Storage Monitoring Commands

```
# Check all filesystem usage:
df -h
# Look for any filesystem above 70%

# Check Proxmox storage pools specifically:
pvesm status
# Shows local and local-lvm usage

# Check LVM thin pool percentage:
lvs
# Look for the data% column — alert if above 75%

# Check external HDD (Wazuh data):
df -h /mnt/hdd
# Critical if above 80%

# Check all running VMs and containers:
qm list && pct list
# Ensure all expected VMs are running
```

Automated Storage Monitoring Script

```
# Create the script:
nano /root/storage-check.sh

#!/bin/bash
echo '===== STORAGE STATUS ====='
echo 'Date:' $(date)
echo ''
echo '— Filesystem Usage —————'
df -h | grep -E '(Filesystem|/|/var|/mnt)'
echo ''
echo '— Proxmox Storage Pools —————'
pvesm status
echo ''
echo '— LVM Thin Pool —————'
lvs | grep data
echo ''
echo '— VM and Container Status —————'
qm list
pct list
echo '===== '

# Make executable:
chmod +x /root/storage-check.sh

# Add to weekly cron (runs Monday at 9am):
crontab -e
# Add:
0 9 * * 1 /root/storage-check.sh >> /var/log/storage-check.log 2>&1
```

Chapter 16 — Alerts & Notifications

CRITICAL Without real-time notifications, you only know about security incidents when you happen to open the Wazuh dashboard. Telegram notifications deliver alerts to your phone within 30 seconds — even while you sleep. This is not optional for effective SOC operation.

16.1 Telegram Bot Setup

Telegram provides a free bot API that Wazuh uses to send alerts directly to your phone. Setup takes approximately 5 minutes:

Step 1 — Create a Telegram Bot

```
# Open Telegram on your phone or desktop
# Search for: @BotFather
# Start a conversation and send: /newbot
# Follow prompts:
#   Bot name: HomSOC Alerts (or any name you like)
#   Bot username: must end in 'bot' (e.g. myhocsoc_bot)

# BotFather replies with your Bot Token:
# Format: 123456789:AABBCCDDeeffGGHHiijjKKLL...
# SAVE THIS TOKEN — you need it for the Wazuh integration
```

Step 2 — Get Your Chat ID

```
# Send any message to your new bot in Telegram
# Then open this URL in a browser (replace TOKEN with your actual token):
# https://api.telegram.org/botTOKEN/getUpdates

# Look in the JSON response for:
# 'chat': { 'id': 123456789 }
# That number is your Chat ID — save it
```

Step 3 — Create Notification Script on Wazuh VM

```
cat > /var/ossec/integrations/telegram << 'SCRIPT'
#!/bin/bash

ALERT_FILE=$1
BOT_TOKEN='YOUR_BOT_TOKEN_HERE'
CHAT_ID='YOUR_CHAT_ID_HERE'

LEVEL=$(cat $ALERT_FILE | python3 -c "
import json,sys
d=json.load(sys.stdin)
print(d['rule']['level'])
")

DESC=$(cat $ALERT_FILE | python3 -c "
import json,sys
d=json.load(sys.stdin)
print(d['rule']['description'])
")

AGENT=$(cat $ALERT_FILE | python3 -c "
import json,sys
d=json.load(sys.stdin)
print(d.get('agent',{}).get('name','unknown'))
")

TIME=$(cat $ALERT_FILE | python3 -c "
import json,sys
```

```

d=json.load(sys.stdin)
print(d['timestamp'])
")

MSG="🚨 HomSOC Alert%0A"
MSG+="Level: $LEVEL%0A"
MSG+="Agent: $AGENT%0A"
MSG+=" $DESC%0A"
MSG+="Time: $TIME"

curl -s -X POST \
  "https://api.telegram.org/bot${BOT_TOKEN}/sendMessage" \
  -d "chat_id=${CHAT_ID}&text=${MSG}" > /dev/null
SCRIPT

# Set correct permissions:
chmod 750 /var/ossec/integrations/telegram
chown root:wazuh /var/ossec/integrations/telegram

```

Step 4 — Add Integration to Wazuh Config

```

nano /var/ossec/etc/ossec.conf
# Add inside <ossec_config>:

<integration>
  <name>telegram</name>
  <hook_url>https://api.telegram.org/botYOUR_TOKEN/sendMessage</hook_url>
  <alert_format>json</alert_format>
  <level>7</level>
  <!-- Alerts at level 7 and above trigger Telegram notification -->
</integration>

# Restart Wazuh Manager:
systemctl restart wazuh-manager

```

Step 5 — Test Notification

```

# From a LAN device, run a port scan to trigger an IDS alert:
nmap -sS [LAN_SUBNET].7

# Wait 30 seconds
# Expected: Telegram message arrives on your phone:
# 🚨 HomSOC Alert
# Level: 10
# Agent: suricata
# Suricata: Port scan or network reconnaissance detected
# Time: 2026-04-20T...

```

16.2 Email Alerts (Optional)

Wazuh also supports email alerts via SMTP. This is optional — Telegram is sufficient and easier to configure. If email alerts are desired, configure under `ossec.conf` with your SMTP server details. Refer to Wazuh documentation at documentation.wazuh.com for current email integration instructions.

Chapter 17 — Security Hardening

After the full system is built and validated, apply these hardening measures. None of these are optional — they prevent basic compromise of the SOC infrastructure itself.

17.1 Credential Rotation

Change all default passwords immediately after installation. Never leave default credentials in place:

```
# Proxmox root password:
passwd root

# pfSense admin password:
# Web UI: System → User Manager → admin → Set Password

# Pi-hole admin password:
pihole -a -p

# Wazuh admin password:
/usr/share/wazuh-indexer/plugins/opensearch-security/tools/wazuh-passwords-tool.sh \
-u admin -p YourNewStrongPassword

# Tailscale: Enable MFA at tailscale.com/admin

# Suricata Ubuntu user:
passwd ubuntu
```

17.2 Non-Negotiable Security Rules

These rules must never be violated. Each one exists because a real failure mode was identified and documented:

CRITICAL RULE 1 — pfSense is the ONLY router and DHCP server. Never configure routing, NAT, or DHCP on any other device. A second DHCP server causes IP conflicts. A second router breaks the inline IDS path.

CRITICAL RULE 2 — Bridge assignment: vmbr0=WAN · vmbr1=LAN client · vmbr2=LAN pfSense-side. NEVER swap these. Swapping vmbr0 and vmbr1 exposes your LAN directly to the internet.

CRITICAL RULE 3 — Pi-hole is the ONLY DNS server given to clients. Never configure a secondary DNS in DHCP settings. A secondary allows Pi-hole bypass when Pi-hole is unavailable.

CRITICAL RULE 4 — Tailscale MUST always be started with --accept-dns=false. Without this flag, Tailscale overrides Pi-hole as DNS for every VPN session. All DNS filtering stops silently.

CRITICAL RULE 5 — Always take a Proxmox snapshot before any configuration change. No snapshot = no safety net. Restore takes seconds. Debugging a broken config takes hours.

CRITICAL RULE 6 — Monitor Wazuh disk weekly. A full disk causes silent SIEM failure — no alerts, no error message, no indication. You are blind without knowing it.

CRITICAL RULE 7 — Suricata NIC offloading must always be disabled. If the disable-offload.service fails, Suricata misses packets silently. Verify with: systemctl status disable-offload.service

CRITICAL RULE 8 — Access point ethernet must connect to a LAN port only — NEVER the WAN/Internet port. The WAN port re-enables the AP's router functions and creates double NAT.

Chapter 18 — Daily Operations Runbook

This runbook defines the regular maintenance tasks required to keep the Home SOC operating correctly. A SOC that is not maintained degrades silently — blocklists become stale, rules become outdated, disks fill up, and agents disconnect.

18.1 Maintenance Schedule

Task	Frequency	How	Why
Review Wazuh security alerts	Daily	<code>https://[LAN_SUBNET].7 → Security Events</code>	Manual review catches subtle patterns automated rules miss
Check disk space	Daily	<code>df -h /mnt/hdd</code> on Wazuh VM	Full disk causes silent SIEM failure with no visible error
Update Pi-hole blocklists	Weekly	<code>pihole -g</code>	New malicious domains registered daily — stale lists miss them
Update Suricata rules	Weekly	<code>sudo suricata-update && systemctl restart suricata</code>	New attack signatures released weekly by Emerging Threats
Verify all agents active	Weekly	<code>/var/ossec/bin/agent_control -l</code>	Disconnected agents are blind spots — no logs from that source
Run health check script	Weekly	<code>/root/storage-check.sh</code>	Catches silent failures across all storage pools
Update Proxmox	Monthly	<code>apt update && apt full-upgrade</code>	Security patches for the hypervisor itself
Update pfSense	Monthly	System → Update (take snapshot first)	Firewall security patches — always snapshot before updating
Take snapshots	Before any change	Proxmox → VM/CT → Snapshots → Take Snapshot	Safety net — restore in seconds if a change breaks something

18.2 Weekly Health Check Commands

Run these commands every week to verify the system is operating correctly:

```
# — On Proxmox Host —————
brctl show
# Expected: vmbr0, vmbr1, vmbr2 all listed

qm list && pct list
# Expected: all VMs and LXC's shown with status 'running'

pvesm status
# Check storage pool usage — alert if any above 70%

# — On Wazuh VM —————
systemctl status wazuh-manager wazuh-indexer wazuh-dashboard
# Expected: all three 'active (running)'

df -h /mnt/hdd
# Expected: below 80% used
```

```

/var/ossec/bin/agent_control -l
# Expected: all agents show 'Status: Active'

# — On Suricata VM —————
systemctl status suricata
# Expected: active (running)

systemctl status disable-offload.service
# Expected: active (exited) – offloading disabled

sudo tail -5 /var/log/suricata/fast.log
# Review recent alerts

# — In Pi-hole LXC —————
pihole status
# Expected: DNS service running, blocking enabled

# — In Tailscale LXC —————
tailscale status
# Expected: device connected, peers listed

```

18.3 Snapshot Strategy

Snapshots are your primary recovery tool. A disciplined snapshot strategy means any mistake is recoverable:

- Before any pfSense configuration change: snapshot VM 100
- Before any Wazuh rule or decoder change: snapshot VM 104
- Before any Suricata configuration change: snapshot VM 101
- Before any Pi-hole change: snapshot LXC 102
- Before any system update (apt upgrade): snapshot all VMs
- After a successful build phase: take a 'known good' snapshot of all components

```

# Take a snapshot (Proxmox console):
qm snapshot <vmid> <snapshot-name> --description '<description>'
# Example:
qm snapshot 100 before-firewall-rules 'Before adding DNS enforcement rules'

# List snapshots:
qm listsnapshot <vmid>

# Rollback to snapshot:
qm rollback <vmid> <snapshot-name>
# Example: qm rollback 100 before-firewall-rules

# For LXC containers, use 'pct' instead of 'qm':
pct snapshot 102 before-blocklist-change
pct rollback 102 before-blocklist-change

```

Chapter 19 — Full Rebuild Checklist

NOTE This checklist contains every step required to rebuild the entire Home SOC from a bare laptop with no operating system. Follow phases in strict order. Validate completely before moving to the next phase. Each validation checkpoint is a gate — do not proceed if it fails.

DETERMINISTIC REBUILD The entire system is defined by the configuration files documented in this manual. Nothing requires undocumented knowledge. A complete rebuild from bare metal to fully operational SOC should take approximately 4-6 hours following this checklist.

Phase 0 — Hardware Preparation

- [] Confirm CPU supports virtualisation: VT-x (Intel) or AMD-V (AMD) — verify in BIOS
- [] Set BIOS boot mode to UEFI
- [] Disable Secure Boot in BIOS
- [] Two network connections ready: built-in Ethernet (WAN) + USB Ethernet adapter (LAN)
- [] USB drive (8GB+) flashed with Proxmox VE 8.x ISO
- [] pfSense CE ISO downloaded and available
- [] Ubuntu Server 22.04 LTS ISO downloaded and available

Phase 1 — Proxmox VE

- [] Install Proxmox to internal SSD from USB (erases all existing data)
- [] Set management IP: [WAN_SUBNET].10/24, gateway [WAN_SUBNET].1, hostname: pve
- [] Fix enterprise repository: comment out pve-enterprise.list
- [] Add no-subscription repository to sources.list
- [] Run: apt update && apt full-upgrade -y
- [] Disable lid sleep: set HandleLidSwitch=ignore in logind.conf, restart logind
- [] Mask sleep targets: systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target
- [] Set GRUB consoleblank=60, run update-grub, reboot
- [] VALIDATE: https://[WAN_SUBNET].10:8006 loads and login works

Phase 2 — Network Bridges

- [] Edit /etc/network/interfaces: create vmbr0 (built-in NIC, WAN), vmbr1 (USB NIC, LAN), vmbr2 (none, internal)
- [] Run: ifreload -a (or reboot for clean apply)
- [] VALIDATE: brctl show lists vmbr0, vmbr1, and vmbr2
- [] VALIDATE: ip a shows [WAN_SUBNET].10/24 on vmbr0, [LAN_SUBNET].100/24 on vmbr1, no IP on vmbr2
- [] VALIDATE: ping [WAN_SUBNET].1 succeeds from Proxmox host

Phase 3 — pfSense VM

- [] Create VM 100: 4 cores, 4GB RAM, 12GB disk, net0=vmbr0 (WAN), net1=vmbr2 (LAN)
- [] Install pfSense CE, assign WAN=vtnet0, LAN=vtnet1
- [] Set LAN IP: [LAN_SUBNET].1/24
- [] Configure DHCP: pool [LAN_SUBNET].10 to [LAN_SUBNET].99
- [] Set DHCP DNS to [LAN_SUBNET].4 ONLY — leave second DNS field empty
- [] Change admin password from default immediately
- [] VALIDATE: LAN-connected device gets IP in [LAN_SUBNET].10-99 range
- [] VALIDATE: Gateway shows [LAN_SUBNET].1, DNS shows [LAN_SUBNET].4
- [] VALIDATE: Internet access works from LAN device (ping 8.8.8.8)
- [] VALIDATE: http://[LAN_SUBNET].1 loads pfSense web UI

Phase 4 — Access Point

- [] Factory reset access point
- [] Set AP LAN IP to [LAN_SUBNET].2
- [] Disable AP's DHCP server
- [] Configure Wi-Fi SSID and WPA2-PSK password
- [] Connect Ethernet from USB NIC to AP LAN port (NOT WAN port)
- [] VALIDATE: Wi-Fi device gets IP from pfSense ([LAN_SUBNET].10-99)
- [] VALIDATE: Gateway [LAN_SUBNET].1 and DNS [LAN_SUBNET].4 on Wi-Fi device

Phase 5 — Pi-hole

- [] Create LXC 102: Debian 13, unprivileged, 2 cores, 1GB RAM, 8GB disk
- [] Network: vmbr1, static IP [LAN_SUBNET].4/24, gateway [LAN_SUBNET].1
- [] Install Pi-hole: `curl -sSL https://install.pi-hole.net | bash`
- [] Set upstream DNS: 1.1.1.1 primary, 8.8.8.8 secondary
- [] Set Pi-hole admin password: `pihole -a -p`
- [] Add all 5 threat intelligence blocklists (OISD, Steven Black, HaGeZi, URLhaus, Phishing Army)
- [] Run: `pihole -g` to compile Gravity database
- [] Add whitelists: `packages.wazuh.com`, `raw.githubusercontent.com`, `login.tailscale.com`, `pool.ntp.org`
- [] VALIDATE: `nslookup doubleclick.net [LAN_SUBNET].4` returns 0.0.0.0
- [] VALIDATE: `nslookup google.com [LAN_SUBNET].4` returns real IP
- [] VALIDATE: pihole status shows DNS running and blocking enabled

Phase 6 — DNS Enforcement

- [] pfSense: add firewall rules — allow DNS to [LAN_SUBNET].4 (port 53), block all other DNS, block DNS-over-TLS (port 853)
- [] pfSense: add NAT port forward — all DNS (port 53) from LAN net NOT going to [LAN_SUBNET].4 → redirect to [LAN_SUBNET].4
- [] pfSense: disable IPv6 on WAN interface
- [] pfSense: disable IPv6 on LAN interface
- [] VALIDATE: `nslookup doubleclick.net 8.8.8.8` returns 0.0.0.0 (bypass intercepted)
- [] VALIDATE: Pi-hole dashboard Query Log shows ALL DNS queries from all devices

Phase 7 — Wazuh SIEM

- [] Create VM 104: 6 cores, 8GB RAM, 80GB disk, vmbr1, q35 machine, Ubuntu 22.04
- [] Set static IP: [LAN_SUBNET].7/24, gateway [LAN_SUBNET].1, DNS [LAN_SUBNET].4
- [] Install Wazuh 4.14.2 all-in-one: `bash wazuh-install.sh -a`
- [] SAVE admin credentials output by installer immediately
- [] Attach external USB HDD to VM 104
- [] Format HDD as ext4, mount at `/mnt/hdd`, add to `/etc/fstab` with `nofail`
- [] Move Wazuh index data to `/mnt/hdd`, update `opensearch.yml` paths
- [] Restart all Wazuh services
- [] Change Wazuh admin password from installer-generated password
- [] VALIDATE: `https://[LAN_SUBNET].7` loads Wazuh dashboard
- [] VALIDATE: All three services active — `wazuh-manager`, `wazuh-indexer`, `wazuh-dashboard`
- [] VALIDATE: `df -h /mnt/hdd` shows HDD mounted and in use

Phase 8 — Suricata Inline IDS

- [] Verify vmbr2 exists: `brctl show`
- [] Move pfSense LAN NIC from vmbr1 to vmbr2 (VM 100 → Hardware → net1 → Edit)
- [] Restart pfSense VM: `qm reboot 100`

- [] Create VM 101: 2 cores, 4GB RAM, 20GB disk, net0=vibr1, net1=vibr2, q35, Ubuntu 22.04
- [] Install: apt install suricata suricata-update bridge-utils ethtool
- [] Create Linux bridge br0 connecting exxxxx8 (vibr1) and exxxxx9 (vibr2)
- [] Assign [LAN_SUBNET].6/24 to br0
- [] Write persistent Netplan config /etc/netplan/00-installer-config.yaml → netplan apply
- [] Disable NIC offloading: create and enable disable-offload.service
- [] Configure suricata.yaml: HOME_NET, af-packet for both interfaces, eve-log output
- [] Download rules: suricata-update
- [] Test config: suricata -T — must show 'successfully loaded'
- [] Start Suricata: systemctl enable --now suricata
- [] Install Wazuh agent, configure eve.json localfile, start agent
- [] VALIDATE: tcpdump -i exxxxx8 shows SYN packets when LAN device browses web
- [] VALIDATE: nmap scan from LAN generates alerts in /var/log/suricata/fast.log
- [] VALIDATE: Suricata agent shows Active in Wazuh: /var/ossec/bin/agent_control -l

Phase 9 — Log Integrations

- [] pfSense: configure remote syslog to [LAN_SUBNET].7:514 (Firewall + System events)
- [] Wazuh VM: add syslog receiver block to ossec.conf for [LAN_SUBNET].1 on UDP 514
- [] Pi-hole LXC: install Wazuh agent, add pihole.log localfile block, start agent
- [] Tailscale LXC: install Wazuh agent, add journald filter block, start agent
- [] Proxmox host: install Wazuh agent, add syslog and pveproxy localfile blocks, start agent
- [] Add all custom decoders to local_decoder.xml
- [] Add all custom rules to local_rules.xml
- [] Restart wazuh-manager
- [] VALIDATE: /var/ossec/bin/agent_control -l shows ALL agents as Active
- [] VALIDATE: Wazuh dashboard shows events from pfSense, Pi-hole, Suricata

Phase 10 — Tailscale VPN

- [] Create LXC 103: Debian 13, unprivileged, 2 cores, 512MB RAM, 8GB disk
- [] Network eth0: vibr1, static [LAN_SUBNET].5/24
- [] Add second NIC eth1: vibr0, DHCP
- [] Add TUN device config to /etc/pve/lxc/103.conf BEFORE starting container
- [] Start container, enable IP forwarding inside container
- [] Install Tailscale: curl -fsSL https://tailscale.com/install.sh | sh
- [] Start Tailscale with: tailscale up --advertise-routes=[LAN_SUBNET].0/24 --accept-dns=false
- [] Authenticate via browser URL provided
- [] Approve subnet routes in Tailscale admin console
- [] Enable MFA on Tailscale account
- [] VALIDATE: tailscale status shows connected
- [] VALIDATE: ping [LAN_SUBNET].1 from outside via Tailscale succeeds

Phase 11 — Windows Endpoint

- [] Download Wazuh Windows MSI from Dashboard → Agents → Deploy New Agent
- [] Install MSI as Administrator, set Manager IP: [LAN_SUBNET].7
- [] Download Sysmon + SwiftOnSecurity config from GitHub
- [] Install: sysmon64.exe -accepteula -i sysmonconfig-export.xml
- [] Add Sysmon eventchannel block to C:\Program Files (x86)\ossec-agent\ossec.conf
- [] Restart agent: net stop WazuhSvc && net start WazuhSvc
- [] VALIDATE: Windows agent shows Active in Wazuh dashboard

Phase 12 — Notifications & Final Validation

- [] Create Telegram bot via @BotFather, save Bot Token
- [] Get Chat ID from getUpdates API call
- [] Create /var/ossec/integrations/telegram script with correct token and chat ID
- [] Add integration block to ossec.conf (level 7+)
- [] Restart wazuh-manager

Final Validation Checklist

- [] Internet works from Wi-Fi device
- [] Wi-Fi device gets IP in [LAN_SUBNET].10-99, gateway [LAN_SUBNET].1, DNS [LAN_SUBNET].4
- [] Pi-hole blocks doubleclick.net (returns 0.0.0.0)
- [] DNS bypass blocked: nslookup doubleclick.net 8.8.8.8 still returns 0.0.0.0
- [] Wazuh dashboard accessible at https://[LAN_SUBNET].7
- [] All agents active: /var/ossec/bin/agent_control -l
- [] pfSense firewall logs visible in Wazuh Discover
- [] Suricata alerts appear in Wazuh after nmap test
- [] Pi-hole DNS queries appearing in Wazuh
- [] Telegram notification received after nmap test
- [] External HDD mounted, below 70% usage
- [] Snapshots taken of all VMs and LXC's at known-good state

Chapter 20 — Disaster Recovery Runbook

NOTE GOLDEN RULE: Restore the most recent snapshot FIRST. Diagnose second. Snapshots exist to eliminate the pressure of diagnosis during an outage. If a snapshot is available, restore it before attempting any other fix.

Print this chapter and keep it physically accessible. In an outage, having the runbook on paper means you can follow it even if the SOC itself is down and you cannot access digital documentation.

DR-1 — Total Host Failure — Full Rebuild Required

Symptoms: Physical laptop fails, SSD corrupted, or Proxmox will not boot.

Resolution: Follow the Phase 0-12 rebuild checklist in Chapter 19 in its entirety.

- Estimated rebuild time: 4-6 hours following the checklist
- All configuration is documented — nothing requires prior knowledge
- After rebuild: restore from external HDD backup if Wazuh historical data is needed

DR-2 — pfSense Failure — Complete Network Outage

Symptoms: No internet from any LAN device. No DHCP assignments. LAN devices cannot reach [LAN_SUBNET].1.

- Step 1: Proxmox → VM 100 → Snapshots → Rollback to most recent snapshot
- Step 2: Start VM 100 if not running: `qm start 100`
- Step 3: Wait 30 seconds for pfSense to fully start
- Step 4: Verify: ping [LAN_SUBNET].1 from another device
- Step 5: Verify: internet works from LAN device

If no snapshot available:

- Reinstall pfSense from ISO → WAN=vtnet0, LAN=vtnet1 (on vmbr2)
- Set LAN IP [LAN_SUBNET].1, DHCP pool .10-.99, DNS [LAN_SUBNET].4
- Re-add firewall rules and NAT redirect from Chapter 6

DR-3 — Pi-hole Failure — DNS Outage

Symptoms: Websites unreachable. DNS queries failing. Devices show DNS errors.

- Step 1: `pct restart 102`
- Step 2: Wait 15 seconds, then test: `nslookup google.com [LAN_SUBNET].4`
- Step 3: If still failing: `pct stop 102` → Snapshots → Rollback → `pct start 102`
- Step 4: Verify: pihole status shows DNS running

TEMPORARY FIX while diagnosing (restores internet without Pi-hole):

```
# pfSense: Services → DHCP Server → LAN
# Temporarily add DNS Server 2: 1.1.1.1
# IMPORTANT: Remove this secondary after Pi-hole is fixed
# Leaving secondary DNS permanently defeats all DNS-based protection
```

DR-4 — Wazuh Failure — Disk Full (Most Common Failure Mode)

Symptoms: Wazuh dashboard inaccessible or blank. No new alerts generated. Agents show disconnected.

- Step 1 — CHECK DISK FIRST: `df -h /mnt/hdd` inside VM 104
- Step 2 — If above 95%: Wazuh Dashboard → Stack Management → Index Management → delete oldest `wazuh-alerts-*` index
- Step 3 — Verify HDD is still mounted: `mount | grep hdd` (should show `/dev/sdb` on `/mnt/hdd`)
- Step 4 — If HDD unmounted: `mount -a` (uses `fstab` entry)

- Step 5 — Restart all services: `systemctl restart wazuh-indexer wazuh-manager wazuh-dashboard`
- Step 6 — Check logs for errors: `tail -50 /var/log/wazuh-indexer/wazuh-cluster.log`

CRITICAL Disk full is the most common cause of Wazuh failure and is completely silent — the dashboard may appear to work but no new events are being indexed and no alerts are generated. Check disk first, always.

DR-5 — Suricata Failure — No IDS Alerts

Symptoms: No Suricata alerts in Wazuh. Port scans go undetected. `fast.log` is empty or not updating.

- Step 1: `sudo systemctl restart suricata`
- Step 2: Test config: `sudo suricata -T -c /etc/suricata/suricata.yaml`
- Step 3: Verify bridge: `brctl show` → should show `br0` with `xxxxxx8` and `xxxxxx9`
- Step 4: Verify IP on bridge: `ip a show br0` → should show `[LAN_SUBNET].6/24`
- Step 5: Check offloading: `systemctl status disable-offload.service` → should be active (exited)
- Step 6: If offloading enabled: `systemctl start disable-offload.service`
- Step 7: If rules corrupt: `sudo suricata-update --force`
- Step 8: Check Wazuh agent: `systemctl status wazuh-agent`

DR-6 — Tailscale — Remote Access Lost

Symptoms: Cannot access SOC remotely. Tailscale app shows SOC offline.

- Step 1: Access Proxmox console locally: `pct enter 103`
- Step 2: Check status: `tailscale status`
- Step 3: If disconnected: `tailscale up --advertise-routes=[LAN_SUBNET].0/24 --accept-dns=false`
- Step 4: Re-approve routes in Tailscale admin: `login.tailscale.com/admin/machines`
- Step 5: If container broken: `pct stop 103` → Snapshots → Rollback → `pct start 103`

DR-7 — Wazuh Receives No Logs From Any Source

Symptoms: Wazuh dashboard shows no new events. All agent tiles show grey/inactive.

- Step 1: Check pfSense syslog: Status → Logs → Settings → confirm `[LAN_SUBNET].7:514`
- Step 2: On Wazuh VM: `netstat -ulnp | grep 514` → should show Wazuh listening
- Step 3: On Wazuh VM: `/var/ossec/bin/agent_control -l` → identify disconnected agents
- Step 4: For each disconnected agent: SSH in and run: `systemctl restart wazuh-agent`
- Step 5: Restart Wazuh Manager: `systemctl restart wazuh-manager`
- Step 6: Wait 60 seconds and check `agent_control -l` again

Emergency IP Reference — Print This Page

Service	IP Address	Access URL / Method	Default User
Proxmox Hypervisor	<code>[WAN_SUBNET].10</code>	<code>https://[WAN_SUBNET].10:8006</code>	root
pfSense Firewall	<code>[LAN_SUBNET].1</code>	<code>https://[LAN_SUBNET].1</code>	admin
Pi-hole DNS	<code>[LAN_SUBNET].4</code>	<code>http://[LAN_SUBNET].4/admin</code>	admin
Tailscale VPN	<code>[LAN_SUBNET].5</code>	<code>pct enter 103</code> (from Proxmox)	root
Suricata IDS	<code>[LAN_SUBNET].6</code>	<code>ssh ubuntu@[LAN_SUBNET].6</code>	ubuntu
Wazuh SIEM	<code>[LAN_SUBNET].7</code>	<code>https://[LAN_SUBNET].7</code>	admin
Access Point	<code>[LAN_SUBNET].2</code>	<code>http://[LAN_SUBNET].2</code>	admin

Chapter 21 — Future Enhancements

NOTE Expand only after every item in the rebuild checklist is validated and the system has operated stably for 30+ days. Premature expansion on an unstable base creates compounding problems.

[HIGH PRIORITY] Suricata IPS Mode

Current state: Suricata operates as an IDS — it detects attacks and generates alerts, but does not block them. An attacker who scans your network will see alerts in Wazuh but will not be blocked from continuing.

Enhancement: Convert Suricata to IPS (Intrusion Prevention System) mode using NFQ (Netfilter Queue). In IPS mode, Suricata drops malicious packets before they reach their destination. Port scans, exploit attempts, and known malware communication are blocked in real time.

Implementation requirement: NFQ mode requires changes to Suricata configuration and iptables rules on the Suricata VM. The inline bridge architecture of V2 already provides the packet control needed — the bridge position is already correct for IPS operation.

Recommendation: Operate in IDS mode for at least 30 days to understand normal traffic patterns and tune rules before enabling blocking. False positives in IPS mode cause service disruptions.

[HIGH PRIORITY] pfBlockerNG

Current state: Pi-hole blocks malicious domains at the DNS layer. A determined attacker who knows a target's IP can bypass DNS entirely by connecting directly to the IP address.

Enhancement: pfBlockerNG is a pfSense package that blocks traffic at the IP reputation layer — before it reaches Pi-hole or any other service. It uses threat intelligence feeds to maintain lists of known malicious IP ranges and blocks all traffic to and from those ranges at the firewall level.

This provides defence-in-depth: Pi-hole blocks by domain name, pfBlockerNG blocks by IP address. Both must be bypassed for a threat to communicate.

Installation: pfSense Web UI → System → Package Manager → Available Packages → search pfBlockerNG → Install.

[HIGH PRIORITY] Telegram Notifications (if not done)

If Chapter 16 has not been configured: configure Telegram notifications immediately after log integrations are working. Without phone alerts, security incidents are only discovered when you happen to open the Wazuh dashboard. This defeats much of the operational value of a SOC.

[MEDIUM PRIORITY] VLANs for Network Segmentation

Current state: All devices share a single flat LAN ([LAN_SUBNET].0/24). A compromised IoT device (smart TV, baby monitor) has network-level access to the Wazuh SIEM, pfSense, and all other SOC infrastructure.

Enhancement: Create separate VLANs for different device categories:

- VLAN 10: SOC infrastructure (Wazuh, Suricata, Pi-hole, Tailscale, Proxmox)
- VLAN 20: Personal computers and trusted devices
- VLAN 30: IoT and untrusted devices (smart TVs, speakers, cameras)
- VLAN 40: Guest Wi-Fi (internet only, no LAN access)

Implementation requires: additional Proxmox bridges (vbr3+), a VLAN-capable managed switch or access point, and pfSense VLAN interface configuration. This is a significant architecture change — take snapshots of everything before starting.

[MEDIUM PRIORITY] Wazuh Active Response

Current state: Wazuh detects attacks and generates alerts, but takes no automated action. An analyst must manually respond to each alert.

Enhancement: Wazuh Active Response allows automatic defensive actions when specific rules fire. Examples:

- Automatically block the source IP in pfSense firewall when Suricata generates a high-severity alert
- Automatically isolate a Windows endpoint from the network when malware is detected
- Send enriched alert details to a ticketing system for tracking

Implementation: Wazuh Active Response is configured in `ossec.conf` using `<active-response>` blocks. pfSense integration requires the pfSense API or SSH-based firewall rule modification.

[MEDIUM PRIORITY] NAS — Network Attached Storage

Enhancement: A dedicated NAS (Network Attached Storage) LXC provides centralised file storage for the SOC — Wazuh rule backups, PCAP captures, incident documentation, and configuration archives.

Implementation: LXC 105, IP `[LAN_SUBNET].3`, with a host-mounted USB disk bind-mounted into an unprivileged container running a Samba or NFS server.

[LOW PRIORITY] Grafana + Prometheus

Current state: Wazuh provides security event dashboards but does not provide infrastructure health metrics — CPU usage over time, RAM trends, disk I/O, network throughput.

Enhancement: Grafana + Prometheus provides beautiful time-series infrastructure dashboards. Prometheus scrapes metrics from all VMs. Grafana visualises them. This is additive — it complements Wazuh without replacing any existing functionality.

[LOW PRIORITY] Dedicated Security VLAN for SOC Infrastructure

Enhancement: Move Wazuh (`[LAN_SUBNET].7`) and Suricata (`[LAN_SUBNET].6`) to a dedicated management VLAN (e.g. `[LAN_SUBNET_MGMT].0/24`). This means a compromised device on the main LAN cannot directly communicate with or attack the SIEM itself.

This is a significant architecture change that requires VLAN implementation (above) as a prerequisite.

APPENDICES

Appendix A — Master IP & VM Reference Table

Note: All specific IP addresses use subnet labels in this public edition. [WAN_SUBNET] = your WAN-side network (internet-facing). [LAN_SUBNET] = your LAN-side private network.

Device / Service	IP Address	VM/CT ID	OS	Role
ISP Router	[WAN_SUBNET].1	—	—	WAN gateway to internet
Proxmox Host	[WAN_SUBNET].10	—	Proxmox VE 8.x	Hypervisor — management UI
pfSense WAN	DHCP from ISP	VM 100	FreeBSD	WAN interface (vtnet0 on vmbr0)
pfSense LAN	[LAN_SUBNET].1	VM 100	FreeBSD	Gateway · DHCP · Firewall (vtnet1 on vmbr2)
Access Point	[LAN_SUBNET].2	Physical	Router firmware	Wi-Fi AP + switch in AP mode only
Pi-hole	[LAN_SUBNET].4	LXC 102	Debian 13	DNS filter · Malware/ad blocker
Tailscale	[LAN_SUBNET].5	LXC 103	Debian 13	VPN subnet router (WireGuard)
Suricata IDS	[LAN_SUBNET].6	VM 101	Ubuntu 22.04	Inline IDS bridge (vmbr1 ↔ vmbr2)
Wazuh SIEM	[LAN_SUBNET].7	VM 104	Ubuntu 22.04	SIEM · Log analysis · Alerting
DHCP Pool	[LAN_SUBNET].10–99	Dynamic	—	Client devices — phones, laptops, IoT
Infra Reserved	[LAN_SUBNET].1–9	Static	—	Never assigned to DHCP pool
vmbr2	Internal bridge only	Bridge	—	Suricata output → pfSense LAN (V2 addition)

Agent types per endpoint:

Endpoint	IP	Agent Type	Log Sources
Windows Laptop	[LAN_SUBNET].xx (DHCP)	Wazuh Windows MSI + Sysmon	Security EventLog, Sysmon, System, Application
Pi-hole LXC	[LAN_SUBNET].4	Wazuh Linux agent	pihole.log, syslog
Suricata VM	[LAN_SUBNET].6	Wazuh Linux agent	eve.json (IDS alerts), syslog
Tailscale LXC	[LAN_SUBNET].5	Wazuh Linux agent	journald (tailscaled.service)
Proxmox Host	[WAN_SUBNET].10	Wazuh Linux agent	syslog, pveproxy/access.log
pfSense VM	[LAN_SUBNET].1	Syslog (UDP 514)	filterlog (firewall events), DHCP events

Appendix B — All Config Files — Ready to Copy

This appendix consolidates every configuration file in the build. Each file is shown in its complete form ready to copy. Replace [WAN_SUBNET] and [LAN_SUBNET] with your actual subnet values. Replace NIC names (eno1, enxXXX, exxxx8, exxxx9) with your actual interface names.

B.1 — /etc/network/interfaces (Proxmox Host — V2 Three Bridges)

```
auto lo
iface lo inet loopback

# WAN Bridge — Built-in NIC (replace eno1 with your built-in NIC name)
auto vmbr0
iface vmbr0 inet static
    address [WAN_SUBNET].10/24
    gateway [WAN_SUBNET].1
    bridge-ports eno1
    bridge-stp off
    bridge-fd 0
    bridge-vlan-aware no

# LAN Client Bridge — USB NIC (replace enxXXXXXXXXXXXX with your USB NIC name)
auto vmbr1
iface vmbr1 inet static
    address [LAN_SUBNET].100/24
    bridge-ports enxXXXXXXXXXXXX
    bridge-stp off
    bridge-fd 0
    bridge-vlan-aware no

# Internal Bridge — NO physical NIC (V2 addition — connects Suricata to pfSense)
auto vmbr2
iface vmbr2 inet manual
    bridge-ports none
    bridge-stp off
    bridge-fd 0
    bridge-vlan-aware no
```

B.2 — /etc/netplan/00-installer-config.yaml (Suricata VM — V2)

```
network:
  version: 2
  renderer: networkd
  ethernets:
    exxxx8:
      dhcp4: no
    exxxx9:
      dhcp4: no
  bridges:
    br0:
      interfaces:
        - exxxx8
        - exxxx9
      addresses:
        - [LAN_SUBNET].6/24
      routes:
        - to: default
          via: [LAN_SUBNET].1
      nameservers:
        addresses:
          - [LAN_SUBNET].4
      parameters:
        stp: false
        forward-delay: 0
      dhcp4: no
```

B.3 — disable-offload.service (Suricata VM — Systemd Unit)

```
# /etc/systemd/system/disable-offload.service

[Unit]
Description=Disable NIC offloading for Suricata inline bridge
After=network.target

[Service]
Type=oneshot
ExecStart=/sbin/ethtool -K xxxxxx8 gro off lro off tso off gso off
ExecStart=/sbin/ethtool -K xxxxxx9 gro off lro off tso off gso off
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

B.4 — /etc/pve/lxc/103.conf Additions (Tailscale TUN Device)

```
# Add to END of /etc/pve/lxc/103.conf on the PROXMOX HOST
# Do this BEFORE starting the Tailscale LXC for the first time

lxc.cgroup2.devices.allow: c 10:200 rwm
lxc.mount.entry: /dev/net/tun dev/net/tun none bind,create=file
```

B.5 — /etc/fstab Addition (External HDD for Wazuh)

```
# Add to /etc/fstab on the Wazuh VM (VM 104)
# Replace YOUR-ACTUAL-UUID with the UUID from: blkid /dev/sdb

UUID=YOUR-ACTUAL-UUID /mnt/hdd ext4 defaults,nofail 0 2
```

B.6 — ossec.conf — Wazuh Manager Additions

```
<!-- Add inside <ossec_config> on the WAZUH VM -->

<!-- pfSense syslog receiver -->
<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>[LAN_SUBNET].1</allowed-ips>
</remote>

<!-- Telegram alert integration -->
<integration>
  <name>telegram</name>
  <hook_url>https://api.telegram.org/botYOUR_TOKEN/sendMessage</hook_url>
  <alert_format>json</alert_format>
  <level>7</level>
</integration>
```

B.7 — ossec.conf — Per-Agent Localfile Additions

```
<!-- SURICATA VM - add inside <ossec_config> -->
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>

<!-- PI-HOLE LXC - add inside <ossec_config> -->
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/pihole/pihole.log</location>
</localfile>

<!-- TAILSCALE LXC - add inside <ossec_config> -->
<localfile>
  <log_format>journald</log_format>
  <location>journald</location>
  <filter>_SYSTEMD_UNIT=tailscaled.service</filter>
</localfile>

<!-- PROXMOX HOST - add inside <ossec_config> -->
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/syslog</location>
</localfile>
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/pveproxy/access.log</location>
</localfile>

<!-- WINDOWS ENDPOINT - add to C:\...\ossec-agent\ossec.conf -->
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

Appendix C — Validation Commands Quick Reference

Print this appendix. Use it during build and troubleshooting to quickly verify each component.

What to Verify	Command	Expected Result
Proxmox bridges (all 3)	brctl show	vmbr0, vmbr1, vmbr2 all listed
Proxmox bridge IPs	ip a grep -A2 vmbr	vmbr0=[WAN].10, vmbr1=[LAN].100, vmbr2=no IP
WAN connectivity	ping -c4 [WAN_SUBNET].1	4 replies from home router
Suricata bridge	brctl show (on Suricata VM)	br0 with exxxxx8 and exxxxx9 as ports
Suricata bridge IP	ip a show br0	[LAN_SUBNET].6/24 on br0
Suricata sees traffic	sudo tcpdump -i exxxxx8 -n tcp	SYN packets visible when LAN device browses web
Suricata alert test	nmap -sS [LAN_SUBNET].7 (from LAN device)	ET SCAN alert in /var/log/suricata/fast.log
Suricata config valid	suricata -T -c /etc/suricata/suricata.yaml	Configuration provided was successfully loaded
Suricata running	systemctl status suricata	active (running)
NIC offloading disabled	ethtool -k exxxxx8 grep receive-offload	generic-receive-offload: off
Offload service active	systemctl status disable-offload	active (exited) — oneshot service completed
Internet works	ping 8.8.8.8	Replies (from any LAN device)
pfSense reachable	ping [LAN_SUBNET].1	Replies
Pi-hole blocks ad domain	nslookup doubleclick.net [LAN_SUBNET].4	Returns 0.0.0.0
DNS bypass blocked	nslookup doubleclick.net 8.8.8.8	Returns 0.0.0.0 (intercepted by NAT)
Pi-hole running	pihole status	DNS service running, Blocking enabled
Wazuh all services	systemctl status wazuh-manager wazuh-indexer wazuh-dashboard	All three: active (running)
All agents active	/var/ossec/bin/agent_control -l	All agents show Status: Active
Wazuh disk space	df -h /mnt/hdd	Below 80% used
Wazuh syslog listening	netstat -ulnp grep 514	0.0.0.0:514 listening
pfSense logs in Wazuh	Wazuh Discover → agent.name:pfSense	filterlog entries visible
Suricata alerts in Wazuh	Wazuh Security Events → agent:suricata	IDS alerts visible after nmap test
Tailscale connected	tailscale status (inside LXC 103)	Node shows connected with Tailscale IP

Remote access working	ping [LAN_SUBNET].1 from outside via Tailscale	Replies through VPN tunnel
Telegram alerts working	Run nmap test → wait 30s for phone alert	Telegram message received on phone

Appendix D — Troubleshooting Quick Reference

Symptom	Most Likely Cause	First Fix
No internet from any LAN device	pfSense VM down or Suricata bridge broken	Check VM 101 bridge (brctl show on Suricata VM), then verify VM 100 is running
LAN clients get no IP address	pfSense DHCP off or vmbr2 path broken	Check pfSense DHCP server is enabled; verify pfSense net1=vmbr2
Suricata sees no traffic (only ARP)	pfSense LAN not connected to vmbr2 — inline path broken	Verify pfSense net1=vmbr2; run brctl show on Suricata VM
Suricata generates no alerts	NIC offloading enabled, or wrong interface in suricata.yaml	Check disable-offload service; verify af-packet has BOTH exxxxx8 AND exxxxx9
Suricata config test fails	YAML syntax error in suricata.yaml	suricata -T -v for detailed error. Check indentation — YAML is whitespace-sensitive
Websites unreachable	Pi-hole LXC stopped or DNS misconfigured	pct restart 102; verify pfSense DHCP DNS=[LAN_SUBNET].4 only
DNS bypass not blocked	NAT redirect rule missing or wrong	Verify Firewall → NAT → Port Forward has redirect rule for port 53
Wazuh dashboard not loading	Disk full or services stopped	CHECK DISK FIRST: df -h /mnt/hdd; then systemctl restart wazuh-*
No logs in Wazuh	Agents disconnected or syslog misconfigured	agent_control -l to find disconnected agents; check pfSense syslog setting
Tailscale DNS bypasses Pi-hole	--accept-dns=false missing from tailscale up	tailscale down && tailscale up with --accept-dns=false flag
vmbr2 missing from brctl show	vmbr2 not in /etc/network/interfaces	Add vmbr2 block to interfaces file, run ifreload -a
br0 disappears after reboot	Netplan config not saved or not applied	Check /etc/netplan/ file exists with br0 config, run netplan apply
Offloading re-enables on reboot	disable-offload.service not enabled	systemctl enable disable-offload.service && systemctl start disable-offload.service
Wazuh agent shows Disconnected	Agent cannot reach Wazuh Manager at [LAN_SUBNET].7	Verify network connectivity; systemctl restart wazuh-agent on affected device
Telegram alerts not arriving	Wrong Bot Token or Chat ID, or integration not in ossec.conf	Test token: curl api.telegram.org/botTOKEN/getMe; verify integration block in ossec.conf

— End of Document —

Home SOC Definitive Build Manual · Author: Samhith Challa · Version 2 · April 2026
Portfolio Document — Public Edition · Network addresses generalised for public release.